Module 4: Supervised Learning
Submodule: 4.1 Linear Regression
Lecture: Linear Regression

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction

Supervised learning is one of the most common approaches in machine learning, where a model is trained on a dataset containing **input-output**

pairs. This section introduces the concept of "supervised" learning and discusses its fundamental aspects.

In this framework, we aim to learn a **prediction** function h that maps **inputs** X to **outputs** Y, denoted as:

$$h : X \longrightarrow Y$$

We then use h on *new* data.

Examples of Input-Output Pairs

Several examples of supervised learning tasks include:

- **Image Classification**: Given an image x, determine whether it contains a cat y.
- **Text Classification**: Given some text x, determine if it contains hate speech y.
- **Structured Data**: Given some structured data x, such as house attributes, predict the price of the house y.

II. Training Set

In supervised learning, we are given a **training set**, which consists of input-output pairs:

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(n)}, y^{(n)})\} \tag{1}$$

where:

- $x^{(i)} \in X$: is an element from the input space X.
- $y^{(i)} \in Y$: is its corresponding label from the output space Y.

III. Finding the Hypothesis Function h

The goal is to find a good h, often called the **hypothesis function**. A good $h : X \longrightarrow Y$ should generalize well to new data.

- The function h is determined using a **training algorithm**.
- Algorithms like **Stochastic Gradient Descent (SGD)** are commonly used.

IV. Generalization to New Data

It is crucial to assess h's ability to generalize to new, unseen data:

- Use a **test set** or a **development set**, which is used to evaluate the model's performance on new data.
- The training process is not just about memorizing the training data but about learning patterns that can be applied to future data.
- As an example, a model trained on a specific set of images should be able to classify newly introduced images correctly.

This is called "prediction." We are *very* interested in x that is not an element of the training set ($x \notin trainingset$).

V. Classification vs. Regression

- If Y is **discrete**, the process is called **classification**.
  - Example: Determining whether a text contains hate speech or not.
- If Y is **continuous**, the process is called **regression**.
  - Example: Predicting house prices based on features.

VI. Example Data: House Prices

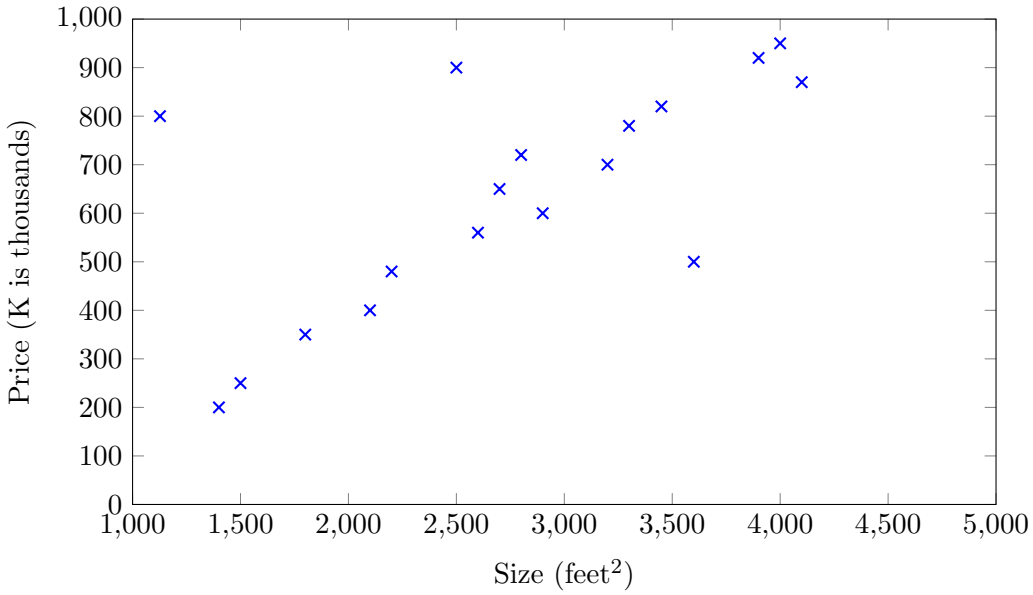
- Regression here is not necessarily broader than the univariate or multivariate versions in statistics. We will sometimes consider high-dimensional regression problems.
- Classification and regression will be used interchangeably in some contexts.
- A regression model that predicts continuous house prices may not perform equally well in classification tasks.
- Example of classification: Predicting whether a house price is above or below \$1,000,000.
- Errors near the classification boundary are critical and should be handled carefully.

Let's examine an example and delve into more details.

Size (feet^2)	Price (K is thousands)
2100	$\$400K$
2500	$\$900K$
1127	$\$800K$
$\vdots$	$\vdots$

Training Set

$h : \text{feet}^2 \longrightarrow price$



Given data like this, how can we learn to predict the prices of houses, as a function of the size of their living areas, that are not in the training data?

VII. Representation of h

We aim to represent h as a function mapping one set to another. Since there are infinite possible functions, we need some restrictions to define a proper representation. The approach: **parameterization** of functions.

To establish notation for future use, we'll use:

- $x^{(i)}$: to denote the input variables (e.g., size in this example), also called input **features**
- $y^{(i)}$: to denote the **output** or **target** variable that we are trying to predict (e.g., price).
- A pair (x, y) is called a **training example** and $(x^{(i)}, y^{(i)})$ is the i^{th} example in $i = 1 \dots n$.
- The list of n training examples $\{(x^{(i)}, y^{(i)}); i = 1 \dots n\}$ is called a **training set**.
- The superscript (i) in the notation is simply an index into the training set and has nothing to do with exponentiation.

When the target variable we're trying to predict is continuous, such as in our housing example, we call the learning problem a **regression** problem. When y can take on only a small number of discrete values (such as if, given the living area, we wanted to predict if a dwelling is a house or an apartment), we call it a **classification** problem.

Affine (Linear) Functions

We focus on **affine (or linear) functions** for representation. In **machine learning**, these are often called linear functions, even though they are technically affine. An **affine function** is simply a linear function plus a constant term.

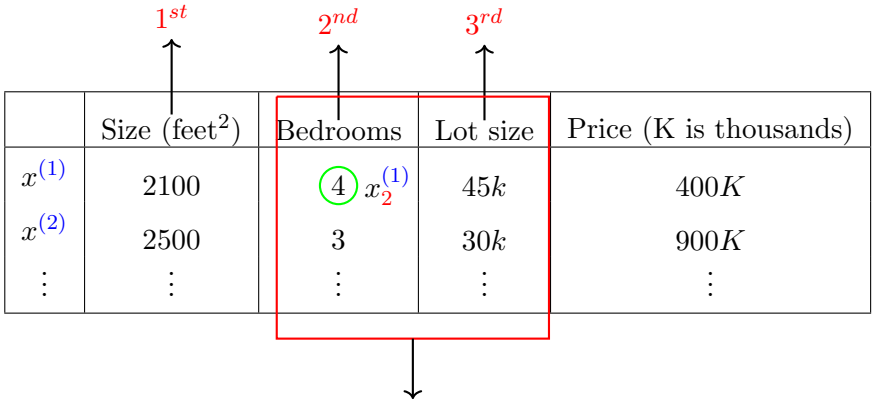
$$h(x) = \theta_0 + \theta_1 x \qquad \text{(affine function)}. \tag{2}$$

Training Set and Data Representation

Maybe we also have other information like the bedrooms, Lot size, lets break down our **training set**:

	Size (feet^2)	Bedrooms	Lot size	Price (K is thousands)
$x^{(1)}$	2100	4	$45k$	\$400K
$x^{(2)}$	2500	3	$30k$	\$900K
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

- Notation:
  - $x^{(i)}$ represents the i-th training example.
  - Each feature is denoted as $x_j^{(i)}$, where j is the feature index.



Here, the $\boldsymbol{x}$'s are three-dimensional vectors ($x \in \mathbb{R}^3$):

- $x_1^{(i)}$ represents the **size** of the i-th house in the training set.
- $x_2^{(i)}$ represents the **number of bedrooms** in that house.
- $x_3^{(i)}$ represents the **lot size** in that house.

Mathematical Representation of Hypothesis h

To perform supervised learning, we must decide how to represent hypotheses $\boldsymbol{h}$ in a computer. As an initial choice, we approximate y as a linear function of $\boldsymbol{x}$:

$$h_\theta(x^{(i)}) = \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \theta_3 x_3^{(i)} = \sum_{j=0}^{d} \theta_j x_j^{(i)}$$

where θ_i are the **parameters** (also called **weights**) that define the linear function mapping from X to Y. Picking those parameters is now the job of the learning function, which we'll discuss. Both θ and $x^{(i)}$ are vectors and $\boldsymbol{d}$ is the number of input features (excluding x_0), also referred to as the dimensionality of the input vector.

To simplify notation, we introduce the convention $x_0 = 1$. x_0 is sometimes referred to as the bias or intercept term. It enables us to express the hypothesis in vector notation.

$$h(x^{(i)}) = \sum_{j=0}^{d} \theta_j x_j^{(i)} = \theta^T x^{(i)}$$

Where θ and $x^{(i)}$ are both vectors of length d:

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_d \end{bmatrix} \qquad x^{(i)} = \begin{bmatrix} x_0^{(i)} \\ x_1^{(i)} \\ \vdots \\ x_d^{(i)} \end{bmatrix}$$

When there is no risk of confusion, we drop the θ subscript and simply write:

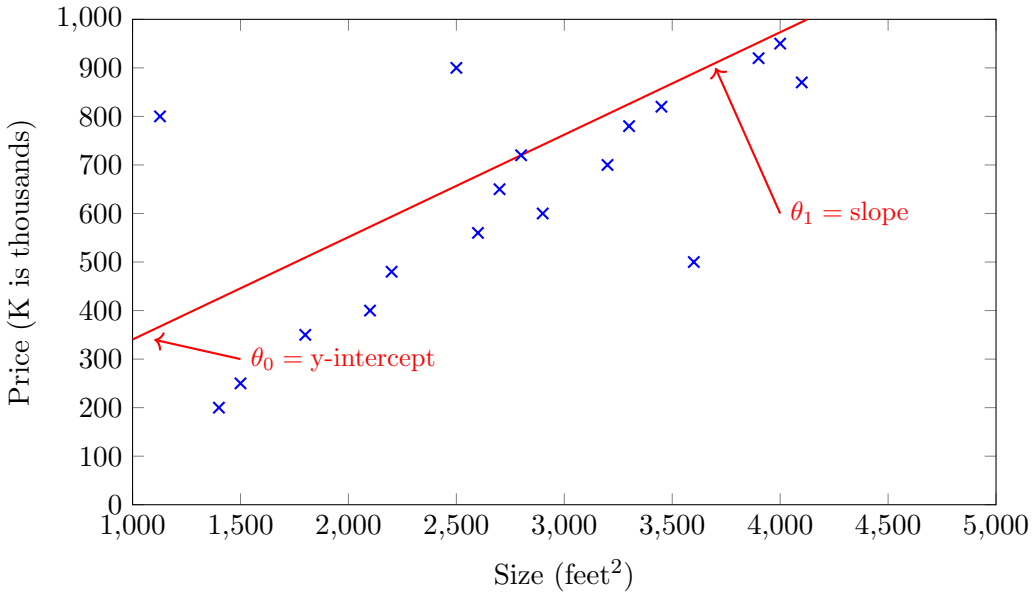
$$h(x) = \theta^T x$$

Additionally, an x without the superscript (i) is frequently used to represent a single input vector.

Given a training set, define the **design matrix** $X \in \mathbb{R}^{(n \times d+1)}$ to be the $n \times d$ matrix (actually $n \times d+1$ if we include the intercept term) that contains the training examples' input values in its rows:

$$X = \begin{bmatrix} — & (x^{(1)})^T & — \\ — & (x^{(2)})^T & — \\ & \vdots & \\ — & (x^{(n)})^T & — \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \dots & x_d^{(n)} \end{bmatrix}$$

Looking at a concrete example, using the data from above, the learning algorithm will pick $h(x) = \theta_0 + \theta_1 x$, which will be a line. θ_0 is the y-intercept and θ_1 is the slope of the line. To predict the value of a new house, we simple plug the sq. ft into x and compute $h(x)$.



VIII. Cost function

Given a training set, how do we determine the parameters θ? A reasonable approach is to make $h(x)$ as close as possible to y for the training examples. In other words, we want to pick θ such that $h_\theta(x) \approx y$.

To formalize this, we define a function that measures how close $h(x^{(i)})$ is to $y^{(i)}$ for all training examples. This function is called the **cost function** or **loss function**:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2.$$

This is the familiar **least-squares cost function**, which gives rise to the **ordinary least squares** linear regression model. The goal is to minimize θ over $J(\theta)$. (The $\frac{1}{2}$ in front of the summation is for convention because that will cancel with the 2 exponent when we take the derivative.)

Even if you have not encountered this function before, we will see later that it is a special case of a much broader class of optimization algorithms.

IX. Common questions

Why Set $x_0 = 1$? Why not choose another constant, like 5 or 0?

- The choice of $x_0 = 1$ ensures that:
  - The bias term θ_0 is included.
  - The scaling remains consistent.
- If another value (e.g., 5) was used, θ_0 would adjust accordingly.

*These functions $h_\theta(x)$ are **affine** because:*

- They include a decision variable θ (parameter).
- Each feature x_j is multiplied by a corresponding weight θ_j.

X. Summary

Supervised learning helps extract patterns from labeled data to make accurate predictions for future inputs.

- **Supervised Learning:** The model learns from labeled data, mapping inputs to outputs.
- **Types of Tasks:**

  - **Classification:** Predicts categorical labels (e.g., spam detection).
  - **Regression:** Predicts continuous values (e.g., house price estimation).

- **Training and Generalization:** A well-trained model should perform well on unseen test data.

- **Model Representation:**
  - A simple linear model:

$$h(x) = \theta_0 + \theta_1 x$$

  - Features can be expanded (e.g., polynomial regression) to improve accuracy.

Module 4: Supervised Learning
Submodule: 4.1 Linear Regression
Lecture: Gradient Descent

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction

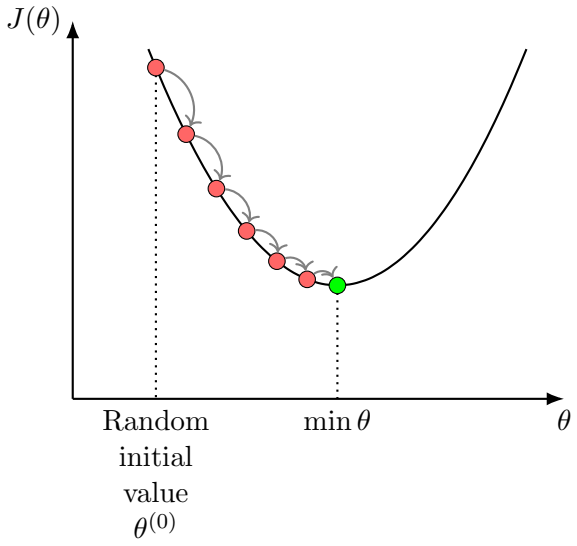
The goal of this discussion is to understand how to find the parameters θ using a method called **gradient descent**. Gradient descent is an iterative optimization algorithm used to minimize a function, particularly in the context of machine learning.

II. Gradient Descent Overview

To find the optimal value of θ that minimizes the function $J(\theta)$, we employ an iterative search algorithm. This algorithm begins with an initial guess for θ and continuously updates it to reduce $J(\theta)$. The process continues until θ converges to a value that minimizes $J(\theta)$. One widely used approach for this optimization is the **gradient descent** algorithm.

Assume we have a cost function $J(\theta)$ that we wish to minimize. Imagine that it appears as illustrated below. (Its important to note that this is a generic representation that may not accurately reflect the specific $J(\theta)$ we

discussed earlier. Our goal here is to understand how the gradient descent algorithm operates in relation to this hypothetical loss function.)



If $J(\theta)$ is convex (bowl-shaped), any local minimum found during the gradient descent process will also be a global minimum. This characteristic is particularly useful in least squares regression since the cost function is convex, which ensures that the solution is optimal.

The process of gradient descent involves the following steps:

1. Initialization - Begin at a random or predefined point (e.g., $\theta^{(0)} := 0$).

2. Iterative Updates - For each iteration t (timestep):

$$\theta_j^{(t+1)} := \theta_j^{(t)} - \alpha \frac{\partial}{\partial \theta_j} J(\theta^{(t)}), \qquad j = 0 \dots d.$$

   α is a positive hyperparameter known as the **learning rate** or **step size**. It adjusts how quickly we traverse the cost curve. There are theories on how to set α which we'll cover later. We will compute the partial derivitate for each "direction" j. (:= means "assignment".)

   The derivative is also called the **gradient** of $J(\theta)$ at $\theta^{(t)}$ which indicates the direction of steepest *ascent*. (We'll be subtracting the gradient in order to *descend* the cost curve.)

$$\text{Gradient} := \frac{\partial}{\partial \theta_j} J(\theta^{(t)}) = \nabla J(\theta^{(t)})$$

In order to implement this algorithm, we have to work out the partial derivative term on the right hand side. We have:

$$\begin{aligned}
\frac{\partial}{\partial\theta_j}J(\theta) &= \sum_{i=0}^{n}\frac{\partial}{\partial\theta_j}\frac{1}{2}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)^2 \qquad \text{by definition} \\
&= 2\cdot\frac{1}{2}\sum_{i=0}^{n}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)\cdot\frac{\partial}{\partial\theta_j}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right) \\
&= \sum_{i=0}^{n}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)\cdot\frac{\partial}{\partial\theta_j}\left(\sum_{m=0}^{d}\theta_m x_m^{(i)}-y^{(i)}\right) \\
&= \sum_{i=0}^{n}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)x_j^{(i)}
\end{aligned}$$

Recall that $h_\theta(x^{(i)}) = \theta_0+\theta_1 x_1^{(i)}+\theta_2 x_2^{(i)}\ldots\theta_j x_j^{(i)} = \sum_{j=0}^{d}\theta_j x_j^{(i)}$. Thus,

$$\theta_j^{(t+1)} := \theta_j^{(t)} - \alpha\sum_{i=0}^{n}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)x_j^{(i)}, \qquad j=0\ldots d \quad (1)$$

The rule is called the **LMS** update rule (LMS stands for "least mean squares"), and is also known as the **Widrow-Hoff** learning rule.

By grouping the updates of the coordinates into an update of the vector θ, We can rewrite update (1) in a slightly more succinct way as:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha\sum_{i=0}^{n}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)x_j^{(i)}, \qquad j=0\ldots d \quad (2)$$

In conclusion, gradient descent is a powerful optimization technique that extends beyond least squares regression, making it adaptable for various machine learning problems. By incrementally updating parameters in the direction opposite the gradient, we can efficiently converge to a minimum of the cost function.

III. Mini-Batch Gradient Descent

In machine learning, we often deal with high volumes of data. When using standard **batch gradient descent**, calculations involve utilizing the entire dataset, which can be time-consuming and inefficient, especially with very large datasets. This method requires examining every single data point to compute a gradient for updating the model, thus making it slow.

To address this issue, the concept of **mini-batch gradient descent** was introduced. This approach randomly selects a subset (mini-batch) of

data points, significantly speeding up the computation while allowing for effective gradient estimation. A mini-batch consists of a small number of points. For example, from a dataset with one million data points, we might select B to be as small as 16 or even just 1. Often B is chosen to optimize training performance based on the amount of GPU memory.

$$B = \{i_1, i_2, \ldots, i_b\}, \quad b < n. \qquad \text{(Training set)}$$

When $B = 1$, meaning we update the parameters with a single training example, the algorithm is referred to as **stochastic gradient descent (SGD)**. There are "intelligent" methods for selecting the examples to be included in a batch which improves performance over selecting randomly.

Updated Gradient Calculation

Instead of calculating the gradient over the entire dataset as in (2), we sum the contributions from the selected mini-batch. The gradient *estimation* can be represented as:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha_b \sum_{k \in B} \left(h_\theta \left(x^{(i)} \right) - y^{(i)} \right) x_j^{(i)}.$$

Note:

- The learning rate used in mini-batch updates α_b might differ from that used in full batch updates α. There is an important connection between the batch size and the learning rate for achieving effective convergence.
- It's beneficial to ensure that each point in the dataset has a chance of being included in some mini-batch, often achieved by shuffling the data.

Trade-offs in Mini-Batch Gradient Descent

Mini-batch gradient descent offers several trade-offs between speed and accuracy:

- Mini-batching tends to converge in terms of steps to reach a solution faster than batch gradient descent because B is significantly smaller than n, leading to fewer computations. However, one downside is that each estimate based on fewer data points may produce noisier gradients, raising concerns about convergence accuracy.
- In practice, many machine learning practitioners utilize mini-batch gradient descent as it assists in managing redundancy in data effectively. Smaller mini-batch sizes can enhance convergence speed, even if they introduce some noise into the model.

Overall, mini-batch gradient descent represents a significant advancement over the traditional batch method, striking a balance between computational efficiency and performance. It has become a standard approach for training large models in the field of machine learning.

IV. Summary

- **Gradient Descent Purpose**: Gradient descent is an iterative optimization algorithm used to find the optimal parameter θ that minimizes a cost function $J(\theta)$. It is fundamental for training machine learning models.
- **Initialization and Iteration**: The algorithm starts with an initial guess for θ and continuously updates it using the derivative (gradient) of the cost function until convergence is reached.
- **Convergence to Local Minima**: With convex cost functions, any local minimum found during iteration will also be a global minimum, ensuring an optimal solution in least squares regression.
- **LMS Update Rule**: The update rule can be succinctly expressed using the Least Mean Squares (LMS) update formula, which incorporates the learning rate α and derivatives of the cost function.
- **Mini-Batch Gradient Descent**: The introduction of mini-batch gradient descent allows for the efficient processing of datasets by only using a random subset (mini-batch) for gradient calculations, improving speed and computational efficiency.
- **Trade-offs of Mini-Batching**: While mini-batching leads to faster convergence, it may also introduce noise due to fewer samples being used to estimate gradients. Practitioners must balance batch size and learning rate for effective model training.
- **Adaptability of Gradient Descent**: Gradient descent is a versatile optimization technique that applies to various machine learning problems, extending beyond just linear regression.

Module 4: Supervised Learning
Submodule: 4.1 Linear Regression
Lecture: Probabilistic Interpretation of Linear Regression

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction

In this lecture, we discuss regression, laying the foundation for our upcoming sessions. This discussion will lead us to explore a larger unification of machine learning models, a significant development in the field about 20 years ago. We will revisit linear regression and provide a probabilistic interpretation for it. This probabilistic perspective is instrumental because it connects various important models in machine learning.

II. Linear Regression

Linear regression involves determining a linear relationship between a set of input training examples $x^{(i)}$ and corresponding outputs $y^{(i)}$.

Given a set of points $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(n)}, y^{(n)})\}$.

Where the feature vectors $x^{(i)}$ exist in $\mathbb{R}^{d+1}$ for $i = 1, 2, \cdots, n$, including a bias term. $y^{(i)} \in \mathbb{R}$, $i = 1, 2, \cdots, n$. The goal is to find the parameter vector $\theta \in \mathbb{R}^{d+1}$ that minimizes the least squares cost function:

$$\theta = \underset{\theta}{\operatorname{argmin}} \sum_{i=1}^{n} \left(y^{(i)} - h_\theta \left(x^{(i)} \right) \right)^2$$

Here, the hypothesis function is defined as $h_\theta(x) = \theta^T x$.

III. Probabilistic Interpretation

In this section, we will discuss how to interpret linear regression from a probabilistic viewpoint.

To model the relationship between input features $x^{(i)}$ and output $y^{(i)}$, we assume:

$$y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)} \tag{1}$$

where $\epsilon^{(i)}$ is an error term that captures either unmodeled effects (such as if there are some features very pertinent to predicting housing price, but that we left out of the regression) or random noise.

Properties of $\epsilon^{(i)}$:

- We assume that the errors ($\epsilon^{(i)}$) are **independently and identically distributed** (**IID**) random variables that follow a **Gaussian distribution** (also called a Normal distribution).
- The errors are presumed to be IID, so:
  - Unbiased: $\mathbb{E}[\epsilon^{(i)}] = 0$.
  - Independent: $\epsilon^{(i)}$ is independent of $\epsilon^{(j)}$ for $i \neq j$, i.e
    $$\mathbb{E}[\epsilon^{(i)} \epsilon^{(j)}] = \mathbb{E}[\epsilon^{(i)}] \cdot \mathbb{E}[\epsilon^{(j)}].$$
  - Gaussian: Follow a Gaussian distribution with $\mu = 0$ and some variance σ^2:
    $$\mathbb{E}\left[\left(\epsilon^{(i)} \right)^2 \right] = \sigma^2,$$
    We can write this as:
    $$\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2).$$
    and the density of $\epsilon^{(i)}$ is given by
    $$p\left(\epsilon^{(i)} \right) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\left(\epsilon^{(i)} \right)^2}{2\sigma^2} \right).$$

From equation (1) we get

$$p\left(y^{(i)}|x^{(i)};\theta\right) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\left(y^{(i)} - \theta^T x^{(i)}\right)^2}{2\sigma^2}\right). \tag{2}$$

Note:

- The notation $p\left(y^{(i)}|x^{(i)};\theta\right)$ indicates that this is the distribution of $y^{(i)}$ given $x^{(i)}$ and parameterized by θ.
- We should not condition on θ in $p\left(y^{(i)}|x^{(i)};\theta\right)$ since θ is not a random variable.
- We can also write the distribution of $y^{(i)}$ as

  $$y^{(i)} \mid x^{(i)};\theta \sim \mathcal{N}\left(\theta^T x^{(i)}, \sigma^2\right).$$

  where picking a θ effectively picks a distribution. σ can be measured by looking at the noise in the outputs y.

IV. Gaussian Distribution

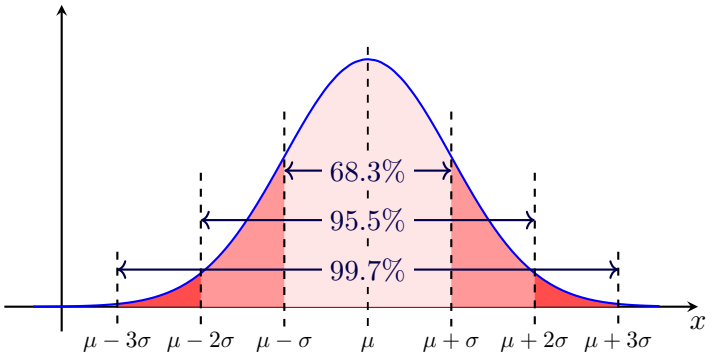
In this section, we delve deeper into the properties and implications of the Gaussian distribution.

The probability density function (PDF) of a Gaussian distribution is defined as:

$$f(y;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y-\mu)^2}{2\sigma^2}}$$

This distribution is characterized by two parameters:

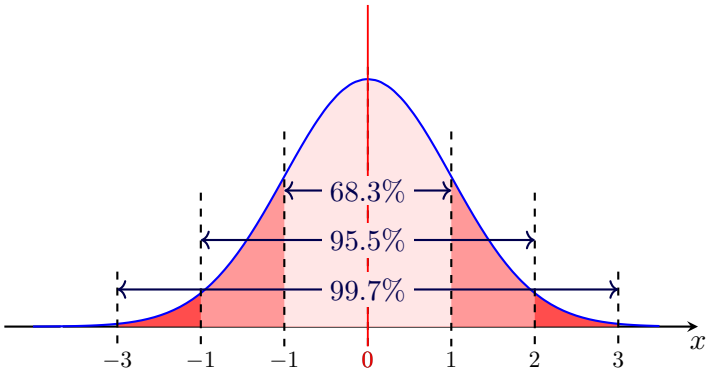
- Mean (μ).
- Variance (σ^2).



Key properties include:

- Symmetry about the mean.
- Total area under the curve equals 1.
- Approximately 68% of the values lie within one standard deviation (σ), 95% within two and 99.7% within three.

Standard normal distribution occurs when a normal random variable has a mean equal to zero ($\mu = 0$) and a standard deviation equal to one ($\sigma = 1$).



V. Likelihood Function

In this section, we will define the likelihood function and its significance.

Given X (the design matrix, which contains all the $x^{(i)}$s) and θ, what is the distribution of the $y^{(i)}$s? The probability of the data is given by $p(\vec{y}|X;\theta)$. This quantity is typically viewed a function of $\vec{y}$ (and perhaps X) for a fixed value of θ. When we wish to explicitly view this as a function of θ, we will instead call it the **likelihood** function.

The **likelihood** $L(\theta)$ of observing the data X given parameters θ is defined as:

$$L(\theta) = p(\vec{y}|X;\theta).$$

Note: Probability and likelihood are both related to the chance of something happening, but they are used in different contexts. Probability quantifies the likelihood of an event occurring, while likelihood is used to assess the plausibility of a parameter value given observed data. In essence, probability is about the uncertainty of an outcome, and likelihood is about the uncertainty of a parameter.

Using the independence assumption on the $\epsilon^{(i)}$'s (and hence also the $y^{(i)}$s given the $x^{(i)}$s), the likelihood can also be written as:

$$L(\theta) = \prod_{i=1}^{n} p\left(y^{(i)} \mid x^{(i)}; \theta\right)$$

From equation (2) we get

$$\begin{aligned} L(\theta) &= p(\vec{y}|X;\theta) \\ &= \prod_{i=1}^{n} p\left(y^{(i)} \mid x^{(i)}; \theta\right) \\ &= \prod_{i=1}^{n} \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\left(y^{(i)} - \theta^T x^{(i)}\right)^2}{2\sigma^2}\right) \end{aligned}$$

The principle of **maximum likelihood** suggests that we should choose the parameter θ in such a way that the likelihood of the data is maximized. In other words, we aim to maximize the likelihood function $L(\theta)$.

Instead of directly maximizing $L(\theta)$, we can maximize any strictly increasing function of $L(\theta)$, which simplifies the derivations. One common transformation is the **log likelihood**, denoted as $\ell(\theta)$. We can express the log likelihood as follows:

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \log \prod_{i=1}^{n} \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\left(y^{(i)} - \theta^T x^{(i)}\right)^2}{2\sigma^2}\right) \\ &= \sum_{i=1}^{n} \log\left(\frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\left(y^{(i)} - \theta^T x^{(i)}\right)^2}{2\sigma^2}\right)\right) \\ &= n \log\left(\frac{1}{\sigma\sqrt{2\pi}}\right) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y^{(i)} - \theta^T x^{(i)}\right)^2 \end{aligned}$$

Maximizing the log-likelihood function $\ell(\theta)$ is equivalent to *minimizing* the following expression:

$$\frac{1}{2} \sum_{i=1}^{n} \left(y^{(i)} - \theta^T x^{(i)}\right)^2$$

This expression is recognized as the least squares cost function $J(\theta)$. So, by maximizing the log-likelihood, we can recover our previously defined loss function related to least squares.

To summarize, under the probabilistic assumptions made on the data, least squares regression can be interpreted as the **maximum likelihood**

estimation (MLE) of the parameter θ. This provides a natural justification for least squares regression as an MLE method.

However, it is important to note that these probabilistic assumptions are not necessary for least squares to be a rational procedure. There are alternative assumptions that can also justify the least squares approach.

The final estimate of θ did not depend on the value of σ^2. This observation remains true even when σ^2 is unknown.

VI. Summary

- **Linear regression**: can be viewed as a maximum likelihood estimation problem under the assumption that the errors are independent, identically distributed, and follow a Gaussian distribution.
- **The likelihood function**: is based on the product of Gaussian distributions for each data point, and the log likelihood simplifies to a sum of squared residuals.
- **Maximizing the likelihood**: is equivalent to minimizing the sum of squared errors, which is the well-known objective in linear regression. This derivation connects the concept of maximum likelihood with the familiar least-squares method for fitting a linear regression model.

Module 4: Supervised Learning
Submodule: 4.2 Logistic Regression
Lecture: Logistic Regression and Classification

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction to Classification

We will discuss the concept of classification, specifically focusing on how we represent data points and their corresponding labels.

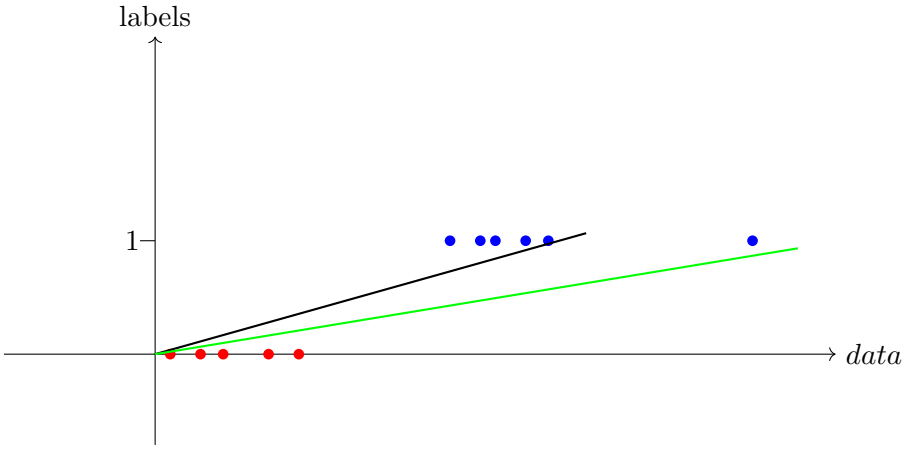
II. Data Representation

Given $\left\{\left(x^{(1)}, y^{(1)}\right) \text{ for } i = 1, \ldots, n\right\}$.

- $x^{(i)} \in \mathbb{R}^{d+1}$, where there are d features plus a bias term
- $y^{(i)} \in \{0, 1\}$, where:
  - $y_i = 0$: negative class (e.g., not a cat or no tumor).
  - $y_i = 1$: positive class (e.g., is a cat or there is a tumor).

III. The Problem with Linear Regression

To visualize the inadequacy of linear regression for classification tasks, consider a one-dimensional feature space.



If we plot the data, we might try to fit a line to separate the classes based on the threshold (e.g., at $y = 0.5$). The issue arises when outliers are present, causing the line (e.g., the green line) to stretch infinitely.

IV. Logistic function

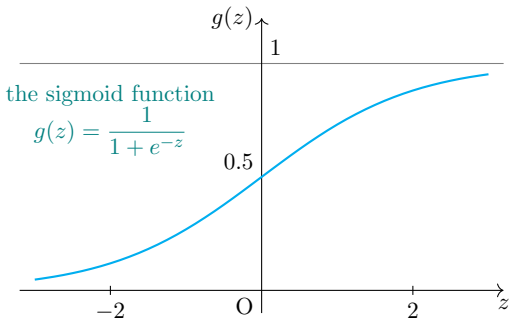
We want our hypothesis h_θ to predict a value between 0 and 1 ($h_\theta(x) \in [0, 1]$) instead of shooting out to infinity. To fix this, let's change the form for our hypotheses $h_\theta(x)$. We will choose

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

where

$$g(z) = \frac{1}{1 + e^{-z}} \quad \text{(link function)}$$

is called the **logistic function** or the **sigmoid function**. Here is a plot showing $g(z)$:



Notice that $g(z)$ tends towards 1 as $z \to \infty$ and $g(z)$ tends towards 0 as $z \to -\infty$. Moreover, g(z), and hence also $h(x)$, is always bounded between 0 and 1.

We interpret $h_\theta(x)$ as the probability that $y = 1$ given x:

$$P(y = 1 \mid x; \theta) = h_\theta(x)$$

And consequently,

$$P(y = 0 \mid x; \theta) = 1 - h_\theta(x)$$

Assuming that the n training examples were generated independently, we can write down the likelihood of the parameters as:

$$\begin{aligned} L(\theta) &= p(\vec{y} \mid \vec{X}; \theta) \\ &= \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}; \theta) \\ &= \prod_{i=1}^{n} \Big(h_\theta(x^{(i)})\Big)^{y^{(i)}} \Big(1 - h_\theta(x^{(i)})\Big)^{1-y^{(i)}} \end{aligned}$$

The $y^{(i)}$ exponents are 'encoding' the "if" condition: if $y^{(i)} = 1$ then the first term is 'active', otherwise it's the second term. Taking the logarithm of the likelihood simplifies calculations:

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^{n} y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)})) \end{aligned}$$

This expression represents the logistic loss, allowing us to proceed with optimization.

V. Maximizing the Likelihood

To maximize the likelihood, we can use a method analogous to the derivation we performed in the case of linear regression. We utilize gradient ascent for this purpose.

Written in vectorial notation, the updates are given by:

$$\theta^{(t=1)} := \theta^{(t)} + \alpha \nabla_\theta \ell(\theta^{(t)})$$

(Note that we use a positive sign in the update formula since we are maximizing rather than minimizing a function.)

Let's start by working with just one training example (x, y) and take derivatives to arrive at the stochastic gradient ascent rule. We differentiate the log-likelihood function $\ell(\theta)$ with respect to the parameter θ_j:

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j}\ell(\theta) &= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right)\frac{\partial}{\partial \theta_j}g(\theta^T x) \\
&= \left(y\frac{1}{g(\theta^T x)} - (1-y)\frac{1}{1-g(\theta^T x)}\right)g(\theta^T x)(1-g(\theta^T x))\frac{\partial}{\partial \theta_j}\theta^T x \\
&= \left(y(1-g(\theta^T x)) - (1-y)g(\theta^T x)\right)x_j \\
&= \left(y - h_\theta(x)\right)x_j
\end{aligned}
$$

In the derivation above, we utilize the fact that the derivative of the logistic function $g(z)$ is given by $g'(z) = g(z)(1-g(z))$.

Thus, we obtain the stochastic gradient ascent rule:

$$
\theta_j^{(t+1)} := \theta_j^{(t)} + \alpha \sum_{i=1}^{n} \left(y_j^{(i)} - h_\theta(x_j^{(i)})\right) x_j^{(i)}
$$

If we compare this update rule with the least mean squares (LMS) update rule, we observe that they appear identical. However, it is crucial to note that this is *not* the same algorithm because $h_\theta(x^{(i)})$ is now defined as a nonlinear function of $\theta^T x^{(i)}$.

Despite the differences in the underlying algorithms and learning problems, it is surprising that we arrive at the same form of the update rule.

The question arises: is this resemblance merely a coincidence, or is there a deeper reason underlying this phenomenon? We will address this intriguing inquiry when we explore generalized linear models (GLM).

1. **Understanding Assumptions**
   - Assumptions are integral components that we introduce to the problem.
   - They enable progress in machine learning.
   - Currently, the assumptions made are relatively simple, primarily concerning functional forms.
2. **Defining Probability Functions**
   - We assume that the probability of y equaling 1 adheres to a specific function.
   - The chosen function is appealing as it:
     - Ranges between 0 and 1,
     - Is monotonically increasing,
     - Possesses certain desirable standard properties.
3. **Complementary Probabilities**

- The reason for using $1 - h_\theta$ is due to the binary nature of y:
- Since y can take only two values ($y \in \{0, 1\}$), the probabilities must sum to 1.

4. **Simple Assumptions and Model Design**

   - The assumptions we make are straightforward linear models or linear models with a specific link function.
   - Discussions can be made on why a particular link function, g, is chosen:
     - Potential exploration of different link functions based on structural properties.

5. **Nature of Assumptions**

   - The assumptions made are not about proving truth; they are models that guide our analysis.
   - By making such assumptions, we can derive significant results.
   - The key question is how well these assumptions reflect the real world.

6. **Independence in Assumptions**

   - The question of when independence matters is nuanced and cannot be definitively answered now.
   - A discussion will be saved for later lessons where independence can be notably violated and affect outcomes significantly.

7. **Modeling Realities**

   - Many variables in the real world are not independent, which can pose challenges.
   - Understanding the limitations of assumptions takes time but can lead to effective modeling practices.

8. **Probabilistic Framework**

   - The transition from full probability to products (due to independence) is critical in the current derivations where we break down data into distinct choices.

VI. Summary

In this lecture, we covered the following key takeaways:

- **Classification Concept:** We explored the fundamentals of classification, emphasizing how data points and their labels are represented.
- **Limitations of Linear Regression:** We identified the issues with using linear regression for classification tasks, particularly in the presence of outliers.
- **Logistic Function:** The logistic function was introduced, allowing us to predict probabilities within the range of [0, 1], which is essential for binary classification.
- **Likelihood Function and Estimation:** We derived the likelihood function for our model, leading to the formulation of the log-likelihood and the concept of logistic loss for optimization.
- **Gradient Ascent:** We explained how to maximize the likelihood using gradient ascent and how this method resembles the update rule used in linear regression.
- **Independence Assumption:** We discussed the assumption of independence in our model and its implications, noting that while it simplifies the problem, it may not hold true in all scenarios.
- **Model Selection:** The importance of evaluating our modeling assumptions in context was emphasized, especially concerning the choice of models and their appropriateness for the data at hand.

Module 4: Supervised Learning
Submodule: 4.3 Newton's Method
Lecture: Newton's Method

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction

In this lecture, we will cover the **Newton's method**. As we build up our vocabulary, there have been phenomenal questions about why use batch gradient descent and why stochastic gradient descent. Newton's method is a powerful and popular closed-form solution to the optimization problem. It was a dominant form of solving these models for a long time. This method is not only interesting but also incredibly beautiful and historical.

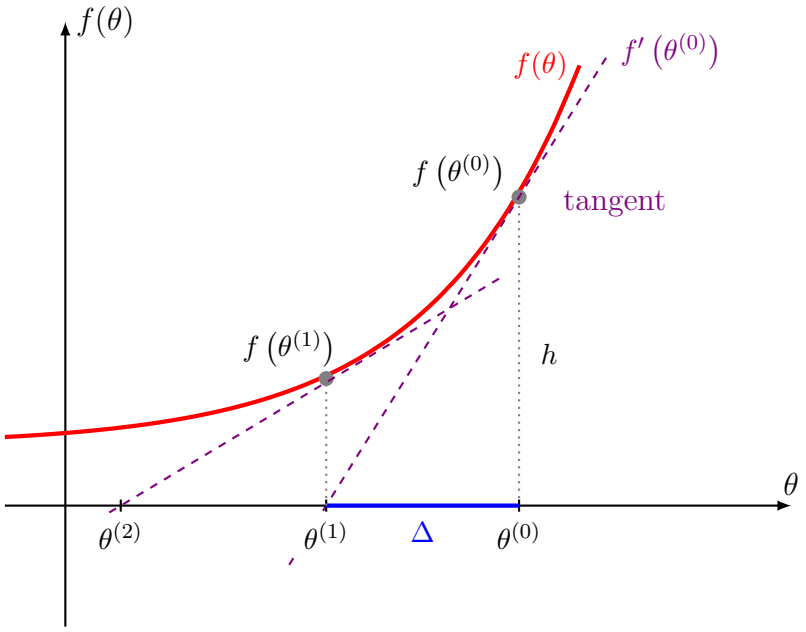
II. Finding Solutions

Our problem is that we are given a function $f : \mathbb{R}^d \to \mathbb{R}$. For illustration purposes, we will look at the simple case of $d = 1$. Our goal is to find x such that $f(x) = 0$.

This is related to machine learning because when we seek to maximize a likelihood fuction (e.g., $\arg\max_\theta \ell(\theta)$), the first-order optimality condition tells us that $\ell'(\theta) = 0$. Therefore, finding the roots of derivatives relates to maximizing or minimizing functions.

III. How Newton's Method Works

We will explore how this method works with a simple function.



Newton's method is an iterative process similar to other methods we have seen before. The following steps elaborate on how it functions:

- Start with an initial guess $\theta^{(0)}$. This is where the method begins.
- Calculate $f(\theta^{(0)})$ to find the value of the function at this point.
- Compute the derivative $f'(\theta^{(0)})$, which represents the slope of the function at $\theta^{(0)}$.
- Newton's method then states that we will follow this line until it intersects the zero point. This will provide us with our next guess. We define this next guess as $\theta^{(1)} = \theta^{(0)} - \Delta$, where Δ represents the distance to be subtracted.
- Calculate the distance Δ based on the idea of rise over run: $f(\theta^{(0)}) = f'(\theta^{(0)}) \cdot \Delta$. This represents the concept that the function value at $\theta^{(0)}$ is equal to the slope at that point multiplied by the distance Δ. Rearranging, this gives us $\Delta = \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$.

This last step leads us to understand Newton's method. Thus,

$$\theta^{(1)} = \theta^{(0)} - \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}.$$

Newtons method performs the following update:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})}.$$

Convergence of Newton's Method

- This method converges locally, meaning it will ultimately reach a root (zero) if one exists.
- The convergence rate is astonishingly fast; it exhibits quadratic convergence. If the first estimate after time T is 0.1, the next step might be 0.01, and even further down to 0.0001. The number of significant digits typically doubles with each iteration:

$$\text{Numbers: } 0.1 \to 0.01 \to 0.0001$$

- Therefore, the error reduces very rapidly. Each step gets you closer to the solution more quickly.
- However, despite its advantages, there are reasons that we don't use Newton's method everywhere: the computation can be quite challenging and resource-intensive.

IV. Generalizing Newton's method

In our logistic regression setting, the parameter vector θ is multidimensional. Thus, we need to generalize Newton's method for this case. The generalized version of Newton's method, often referred to as the **Newton-Raphson method**, is defined by the following equation:

$$\theta := \theta - H^{-1}\nabla_\theta \ell(\theta).$$

Where:

- $\nabla_\theta \ell(\theta)$ is the vector of partial derivatives of the log-likelihood function $\ell(\theta)$ with respect to the components of the parameter vector θ.
- H is a $(d+1) \times (d+1)$ matrix known as the **Hessian**. This matrix is composed of second-order partial derivatives, and its entries are given by:

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j} \in \mathbb{R}^{(d+1)\times(d+1)}$$

In summary, the method updates the parameter vector θ by subtracting the product of the inverse of the Hessian and the gradient of the log-likelihood, allowing for efficient optimization in a multidimensional parameter space.

V. Rough comparison of Optimization Methods

In this section, we will clarify the differences between various optimization methods and discuss the rationale behind selecting each one. It is crucial to understand how these methods operate in order to conduct a meaningful comparison. This discussion goes beyond mere technical details; it represents a conceptual shift that highlights the intersection of traditional optimization techniques with the evolving landscape of machine learning. By grasping these distinctions, we can make informed choices about which methods are best suited to address our current challenges effectively.

Method	Per iteration	Compute	Steps to error ϵ
SGD	1 data point	$O(d)$	ϵ^{-2}
Batch SGD	n data points	$O(nd)$	$\approx \epsilon^{-1}$
Newton's	n data points	$O(nd^2)$	$\log\left(\frac{1}{\epsilon}\right)$

- **Stochastic Gradient Descent (SGD):** SGD takes an individual data point or a small mini-batch at each iteration. The computation required is proportional to the dimension of the problem, resulting in $O(d)$ complexity. It also requires approximately ϵ^{-2} iterations to reach a given error tolerance ϵ.

- **Batch Stochastic Gradient Descent:** Batch SGD computes on all data points, requiring calculations proportional to the number of data points n and the dimensionality d, giving it a complexity of $O(nd)$. It converges a bit more slowly than SGD, requiring about ϵ^{-1} iterations.

- **Newton's Method:** Newton's Method also considers all n data points but it has a complexity of $O(nd^2)$ because it computes the Hessian. It converges logarithmically, taking only $\log\left(\frac{1}{\epsilon}\right)$ steps to reach the error tolerance ϵ. This method is exceptionally fast given that it provides significant digits with each iteration.

These differences matter significantly in the context of classical statistics versus modern machine learning. In classical statistics, n and d are typically small, making batch methods quite effective. However, in machine learning,

both n and d are often extremely large. For example, models like GPT-3 have 175 billion parameters, which makes the computation of the Hessian (requiring roughly 175 billion squared computations) practically impossible for methods like Newton's.

When training large models, modern strategies limit the number of passes over the data (often tens rather than millions), making SGD a better choice when n is large.

In summary, while traditional methods like Newton's provide exact answers and fast convergence under ideal circumstances, SGD and its stochastic nature become more relevant in high-dimensional and large-dataset scenarios, as is common in machine learning today.

VI. Questions

- **Question**: How can we binary classify time series data?

  Answer: Binary classification involves categorizing data into two distinct categories (e.g., yes/no). If the target variable (Y) has more than two classes, a different prediction approach is needed. This session focuses on binary classifications, not multiclass scenarios (like identifying a cat, dog, car).

- Gradient and Hessian
  - The gradient is a vector that represents the direction of the steepest ascent of a function.
  - The Hessian is the second derivative of the loss function, giving us information about the curvature of the function.
  - For a loss function, the Hessian is a matrix derived from the gradient, providing insights into how the loss changes near a point in a multidimensional space.

- Multiclass Classification
  - Multiclass classification goes beyond binary scenarios (like cat, dog, horse, car) and will be discussed in greater detail in the upcoming lecture.
  - Logistic regression can be extended to handle multiple classes through strategies like "one versus all," which breaks down a multiclass problem into multiple binary problems.

- Generalizing Functions and Newton's Method
  - Newton's method involves finding roots of functions. When applied to machine learning, it simplifies the minimization process of loss functions.

  - By generalizing to multiple $(d+1)$ dimensions, we incorporate matrix representations for derivatives, maintaining the relationships of gradients and Hessians.

- Probabilistic Interpretations
  - A theme in the course is utilizing a probabilistic interpretation to derive models like linear regression and logistic regression.
  - This approach helps unify the understanding of classification and regression tasks, indicating that while they may appear distinct at first, they share fundamental underlying mechanisms.

- Convergence and Optimization
  - Newton's Method can find local minima, which raises concerns about potentially missing global minima. The challenge exists due to the nature of the functions. Convex functions guarantee that a local minimum is also global.
  - Modern machine learning often deals with non-convex functions, complicating the optimization process.

- Modeling Error Distributions
  - Beyond Gaussians, various distributions can model errors effectively. The upcoming lectures will cover exponential family distributions, showcasing their relevance across various modeling scenarios.
  - Understanding different assumptions about model errors will be critical throughout the course, especially as it applies to supervised and unsupervised learning.

VII. Summary

In this lecture on Newton's Method, we've covered several important concepts:

- **Newton's Method:** A powerful iterative technique for finding roots of real-valued functions which is particularly useful in optimization problems within machine learning.

- **Convergence Rate:** Newton's Method exhibits quadratic convergence, meaning that the number of correct digits roughly doubles with each iteration, leading to rapid error reduction.

- **Generalization:** We extended Newton's Method to multidimensional spaces using the Newton-Raphson method. The parameter vector is updated by subtracting the product of the inverse Hessian matrix and the gradient of the log-likelihood function.
- **Comparison of Optimization Methods:** We discussed the differences between Stochastic Gradient Descent (SGD), Batch Stochastic Gradient Descent, and Newton's Method, highlighting their computational complexities and efficiencies under different scenarios.
- **Importance of Selection:** In high-dimensional or large dataset contexts, SGD's efficiency makes it preferable, while Newton's provides significant advantages in smaller, more manageable problems.
- **Connection to Machine Learning:** The principles imbued in these optimization methods serve as fundamental tools that enhance both classical statistics and modern machine learning practices, facilitating better model training and prediction accuracy.

Module 4: Supervised Learning
Submodule: 4.4 Generalized Linear Models
Lecture: Exponential Family Models

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction to Exponential Family Models

Exponential family models are an important topic in our lecture today. These models are particularly interesting because they provide a powerful framework for understanding various machine learning models such as regression and classification.

This unification allows us to apply similar techniques across different models repeatedly, enhancing our understanding and making our learning process more efficient.

The main goal of this lecture is to unify inference and learning processes for a variety of models. By decoupling the model's components, we can understand how to derive insights and make predictions effectively.

II. Definitions

The exponential family consists of a specific set of probability distributions. Its significance lies in its broad applicability across various fields. A probability distribution $P(y;\eta)$ is defined in the **exponential family** as follows:

$$p(y;\eta) = b(y)\exp(\eta^T T(y) - a(\eta)).$$

Where:

- y: the data.
- η: are the natural parameters (also called the canonical parameters) of the distribution.
- $T(y)$: is the sufficient statistic, which condenses all the relevant information about the data y into a simplified form. (For the distributions we consider, it will often be the case that $T(y) = y$).
- $b(y)$: is a base measure that outputs a scalar and only depends on the data and does *not* depend on the natural parameters η.
- $a(\eta)$: is a log partition function that outputs a scalar and does *not* depend on the data y.

The quantity $e^{-a(\eta)}$ essentially plays the role of a normalization constant, making sure the distribution $p(y;\eta)$ sums/integrates over y to 1.

It is important to note that not every distribution fits into this framework, but many common distributions do.

The interactions between these components are significant. For instance, $T(y)$ and η must have the same dimension since we're taking the dot product of the two. This means that the dimensionality of the sufficient statistics must align with the natural parameters for valid operations.

III. Examples

Bernoulli Distribution Example

The **Bernoulli distribution** is defined as:

$$P(y;\phi) \stackrel{\triangle}{=} \phi^y(1-\phi)^{1-y}$$

To express the function in an exponential form, we take the logarithm:

$$\begin{aligned} p(y;\phi) &= \phi^y(1-\phi)^{1-y} \\ &= \exp(y\log\phi + (1-y)\log(1-\phi)) \\ &= \exp\left(\left(\log\left(\frac{\phi}{1-\phi}\right)\right)y + \log(1-\phi)\right) \end{aligned}$$

Now, let's identify each component of the Bernoulli distribution in the context of the exponential family. Identifying the components:

- $T(y) = y$
- $\eta = \log\left(\frac{\phi}{1-\phi}\right)$
- $b(y) = 1$
- $a(\eta) = -\log(1-\phi)$

Claim: $a(\eta) = -\log(1-\phi)$.

Proof. We start with the expression for the natural parameter η:

$$\eta = \log\left(\frac{\phi}{1-\phi}\right)$$

We can rewrite ϕ as follows:

$$\phi = \frac{1}{1+e^{-\eta}}$$

Now, we calculate $1-\phi$:

$$1-\phi = 1 - \frac{1}{1+e^{-\eta}} = \frac{e^{-\eta}}{1+e^{-\eta}}$$

Rearranging gives us:

$$1-\phi = \frac{1}{1+e^{\eta}}$$

From the expression $1-\phi = \frac{1}{1+e^{\eta}}$, we take the logarithm:

$$-\log(1-\phi) = -\log\left(\frac{1}{1+e^{\eta}}\right)$$

Applying logarithm rules yields:

$$-\log(1-\phi) = \log(1+e^{\eta})$$

Thus, we have shown that:

$$a(\eta) = -\log(1-\phi) = \log(1+e^{\eta}).$$

□

Gaussian Distribution Example

Recall that, when deriving linear regression, the value of σ^2 had no effect on our final choice of θ and $h_\theta(x)$. Thus, we can choose an arbitrary value for σ^2 without changing anything. To simplify the derivation below, let's set $\sigma^2 = 1$.

Thus, the **Gaussian distribution**, assuming a fixed variance ($\sigma^2 = 1$), is given by:

$$p(y;\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y-\mu)^2\right)$$

We can rewrite this in the exponential family form:

$$\begin{aligned} p(y;\mu) &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2 + \mu y - \frac{1}{2}\mu^2\right) \\ &= \frac{1}{\sqrt{2\pi}} \exp^{(-\frac{1}{2}y^2)} \cdot \exp\left(\mu y - \frac{1}{2}\mu^2\right) \end{aligned}$$

Thus, we see that the Gaussian is in the exponential family, with the components:

$$\begin{aligned} \eta &= \mu \\ T(y) &= y \\ a(\eta) &= \frac{\mu^2}{2} = \frac{\eta^2}{2} \\ b(y) &= \frac{1}{\sqrt{2\pi}} \exp^{(-\frac{1}{2}y^2)} \end{aligned}$$

IV. Why Exponential Family Models Matter

One significant advantage of the exponential family is the simplification it brings to inference:

$$\mathbb{E}[y,\eta] = \frac{\partial a(\eta)}{\partial \eta}$$

This equation shows that the expectation can be computed by the derivative of the log partition function.

The maximum likelihood estimation (MLE) procedure becomes more tractable because the function representing the likelihood is concave. This means that minimizing the negative log likelihood leads to a convex optimization problem, allowing for easier calculation of parameter estimates.

Convex functions have a unique property where every local minimum is also a global minimum, ensuring that we can confidently find optimal model parameters.

V. Summary

In this lecture, we covered essential aspects of exponential family models, highlighting their importance in machine learning applications. Key takeaways include:

- **Definition of Exponential Family:** We defined the exponential family of distributions, emphasizing their structure and components: the natural parameter η, the sufficient statistic $T(y)$, the base measure $b(y)$, and the log partition function $a(\eta)$.
- **Examples:**
  - The **Bernoulli Distribution** was analyzed, where we identified its components and demonstrated a proof for the function $a(\eta) = -\log(1-\phi)$.
  - The **Gaussian Distribution** was examined, illustrating how it fits within the exponential family framework with fixed variance and specifying its parameters.
- **Importance of Inference:** We highlighted the simplification of inference through the relationship between the expectation and the derivative of the log partition function, leading to convenience in maximum likelihood estimation (MLE) procedures.
- **Convexity in Learning:** The significance of convex optimization in parameter estimation was discussed, ensuring unique solutions for optimal parameters, which simplifies the learning process.

Module 4: Supervised Learning
Submodule: 4.4 Generalized Linear Models
Lecture: Generalized Linear Models (GLM)

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction

In this lecture, we discuss the connection between inference and hypothesis functions, linking them to **generalized linear models (GLMs)**. We aim to clarify how these concepts relate to our previous work with models such as logistic regression.

II. Structure of Generalized Linear Models

In this section, we will we will describe a method for constructing GLM models for problems such as these.

III. Design Choices and Assumptions

Here, we will clarify how design choices impact model creation. More generally, consider a classification or regression problem where we would like to predict the value of some random variable y as a function of x. To derive a GLM for this problem, we will make the following three assumptions (and these are not necessarily right or wrong) about the conditional distribution of y given x and about our model:

1. $y \mid x; \theta \sim \text{ExponentialFamily}(\eta)$. ($\sim$ means "behaves according to...") Given x and θ, the distribution of y follows some exponential family distribution, with parameter η.

   Examples include:

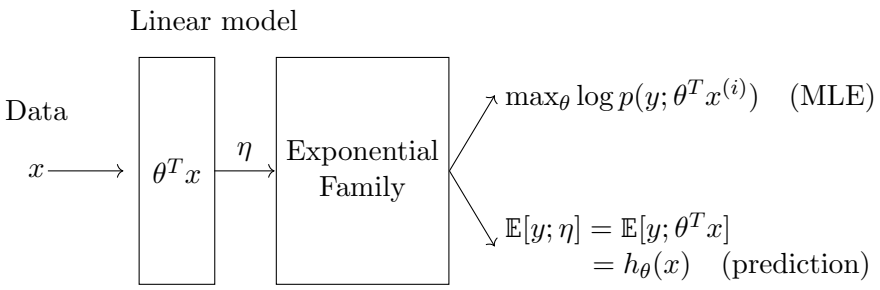
   - **Binary** $y \longrightarrow$ Bernoulli distribution (logistic regression)
   - **Real** $y \longrightarrow$ Gaussian distribution (linear regression)
   - **Count data** $y \longrightarrow$ Poisson distribution
   - **Positive data** $y \longrightarrow$ Gamma or Exponential distribution
   - **Infinite dimensional crazy distribution structure** $y \longrightarrow$ Dirichlet distribution

2. The natural parameter η and the inputs x are related linearly:

   $$\eta = \theta^T x \quad \text{where} \quad \theta \in \mathbb{R}^d, x \in \mathbb{R}^d$$

   (Or, if η is vector-valued, then $\eta_i = \theta_i^T x$.)

3. Given x, our goal is to predict the expected value of $T(y)$ given x. In most of our examples, we will have $T(y) = y$, so this means we would like the prediction $h(x)$ output by our learned hypothesis h to satisfy $h(x) = \mathbb{E}[y|x;\theta]$. (Note that this assumption is satisfied in the choices for $h_\theta(x)$ for both logistic regression and linear regression. For instance, in logistic regression, we had $h_\theta(x) = p(y = 1|x;\theta) = 0 \times p(y = 0|x;\theta) + 1 \times p(y = 1|x;\theta) = \mathbb{E}[y|x;\theta]$.)



IV. Terminology in Generalized Linear Models

In this section, we will explore important terminology and concepts related to GLMs. We will discuss model parameters, natural parameters, canonical parameters and link functions. Furthermore, we will understand the flow of these concepts through examples of logistic and linear regression.

1. **Model Parameters (θ)**: Model parameters, represented by θ, are the coefficients that we estimate during the training of linear and logistic regression models. These are what we train on when we fit a model to our data. The model takes incoming data x and processes it through the equation $\theta^T x$, allowing us to make predictions based on inputs and learned weights.

2. **Natural Parameter (η):** The natural parameter, denoted as η, is derived from the model parameters. Specifically, in generalized linear models, we define it as $\eta = g(\theta^T x)$, where g is a link function that transforms the linear combination of features into a suitable format for modeling the response variable.

3. **Canonical Parameters:** Canonical parameters refer to specific parameters for common probability distributions:

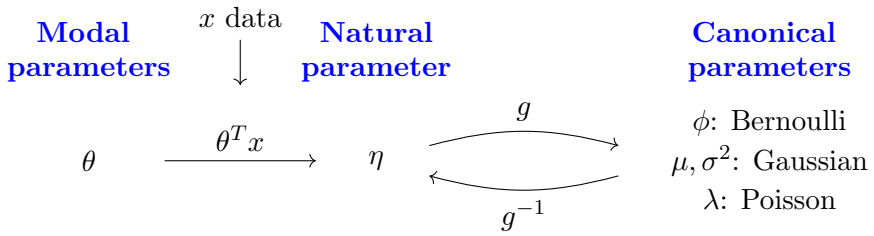
    - **Bernoulli**: For a Bernoulli distribution, the canonical parameter is ϕ, representing the probability of success.
    - **Gaussian**: For the Gaussian distribution, the canonical parameters are μ (mean) and σ^2 (variance).
    - **Poisson**: For the Poisson distribution, the canonical parameter is λ, representing the rate of occurrence.

The **link function** g is used to connect the model's linear predictor ($\theta^T x$) to the expected value of the response variable. Its role is critical in relating model parameters to canonical parameters. There is also an inverse link function g^{-1}, which transforms back from the natural parameter η to the canonical parameters (ϕ, μ, λ).

The function g giving the distribution's mean as a function of the natural parameter

$$g(\eta) = \mathbb{E}[T(y); \eta]$$

is called the **canonical response function**. Its inverse, g^{-1}, is called the **canonical link function**. Thus, the canonical response function for the Gaussian family is just the identify function; and the canonical response function for the Bernoulli is the logistic function.



Flow of Concepts

Now, we will discuss the flow of concepts, illustrating how we move from model parameters to natural parameters and then to canonical parameters.

1. **Model Parameters to Natural Parameters:**

   The journey begins with the model parameters (θ), which are estimated through training. Once we have θ, we can input our data x into the equation $\theta^T x$. This result is then fed into the link function g to yield the natural parameter η.

2. **Natural Parameters to Canonical Parameters:**

   After obtaining η, we can apply the inverse link function g^{-1} to derive the canonical parameters. This connection allows us to understand our model in terms of well-established statistical distributions. For example, if we have derived η from our model parameters, we can use g^{-1} to find the corresponding canonical parameters like ϕ, μ, and λ.

V. Examples

Next, we will illustrate these concepts with practical examples of logistic and linear regression, demonstrating how they apply.

Logistic Regression

In the context of logistic regression, the predicted value $h_\theta(x)$ relates to the probability of an event occurring. Using the logistic link function, we can express the probability of success (a Bernoulli variable $y|x;\theta \sim \text{Bernoulli}(\phi)$) in terms of the natural parameter η. The natural parameter η serves as an intermediary that helps us establish the connection to the canonical parameter ϕ, which represents the probability.

Note that if $y|x;\theta \sim \text{Bernoulli}(\phi)$, then $\mathbb{E}[y|x;\theta] = \phi$. Furthermore, in our formulation of the Bernoulli distribution as an exponential family

distribution, we had $\phi = 1/(1 + e^{-\eta})$. So, we have

$$\begin{aligned} h_\theta(x) &= \mathbb{E}[y|x;\theta] \\ &= \phi \\ &= \frac{1}{1+e^{-\eta}} \\ &= \frac{1}{1+e^{-\theta^T x}}. \end{aligned}$$

Where $\mathbb{E}[y|x;\theta] \in [0,1]$.

How do we use this for classification?

$$\begin{aligned} h_\theta(x) > 0.5 &\implies 1 \\ \text{otherwise} &\implies 0. \end{aligned}$$

Linear Regression

In linear regression, the model's output is directly $\theta^T x$, which corresponds to the mean of the underlying distribution (Gaussian $\mathcal{N}(\mu, \sigma^2)$). This means the natural parameters and model parameters align. Thus, while we operate with model parameters during training, we simultaneously understand them as representing the mean of our Gaussian distribution.

As we saw previously, in the formulation of the Gaussian as an exponential family distribution, we had $\mu = \eta$. So, we have:

$$\begin{aligned} h_\theta(x) &= \mathbb{E}[y|x;\theta] \\ &= \mu \\ &= \eta \\ &= \theta^T x. \end{aligned}$$

VI. Summary

In this section, we will summarize the key takeaways from the lecture.

- Generalized linear models (GLMs) offer a flexible framework for modeling various types of response variables, including binary, continuous and count data.
- The structure of a GLM involves defining model parameters (θ), natural parameters (η), and canonical parameters, which relate to specific probability distributions.
- Understanding the link function is crucial, as it connects the linear predictor to the expected value of the response variable.

- The flow of concepts starts from model parameters, transitions to natural parameters, and then to canonical parameters, illustrating the relationships between these components.
- Practical examples, such as logistic and linear regression, demonstrate how these theoretical ideas are applied in real-world scenarios for classification and prediction tasks.

In conclusion, the generalized linear model provides a robust framework for linking various statistical properties with model parameters. By defining relationships through natural parameters and canonical parameters, inference becomes a clear process.

Module 4: Supervised Learning
Submodule: 4.5 Softmax Regression
Lecture: Multiclass Classification with Softmax Regression

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I. Introduction

In this Lecture, we will introduce the concept of **multiclass softmax regression**, discussing its utility in classification tasks. We will also explain the basic idea of one-hot encoding.

II. Multiclass Softmax

The **multiclass softmax regression** allows for handling multiple discrete outcomes, which can be modeled using a multinomial distribution.

The multinomial distribution expresses probabilities across multiple classes. For instance, in a task where you need to classify images as {'cat', 'dog', 'car', 'bus'} we define $K = 4$ corresponding to these classes.

One-Hot vector

One-hot vector is a crucial concept in transforming categorical data into a numerical format suitable for machine learning. In this representation, each class is represented as a binary vector of length K (i.e., $y \in \{0,1\}^k$).

Examples, in $K = 4$:

- For 'cat': $\mathbf{y}_{\text{cat}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ is class 1.
- For 'dog': $\mathbf{y}_{\text{dog}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$ is class 2.
- For 'car': $\mathbf{y}_{\text{car}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ is class 3.
- For 'bus': $\mathbf{y}_{\text{bus}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$ is class 4.

The one-hot encoding ensures that there is only a single '1' present in any vector, which indicates the true class.

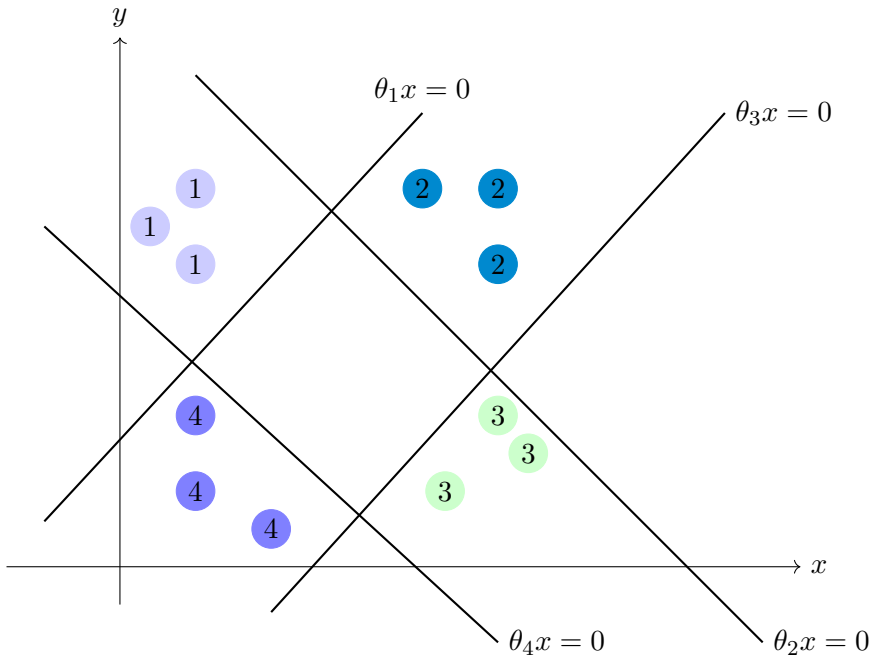
III. Multiclass Classification

In this section, we will explore how to achieve multiclass classification by utilizing hyperplanes and the concept of softmax.

To understand how multiclass classification works, let's consider an example dataset, represented as clusters of points in a feature space. We can visualize the data points and label them according to their respective classes. The colors used to distinguish classes are merely for visual assistance. In multiclass classification, our primary objective is to find hyperplanes that separate each class from the others.

Defining Hyperplanes

The concept of hyperplanes can be visually represented in a two-dimensional space. The equations of the hyperplanes that separate classes will provide a good visual understanding. The diagram below illustrates this idea:



The diagram illustrates different classes of data points represented in a two-dimensional feature space. The colored regions are indicative of the clusters, while the hyperplanes separate these classes.

For each hyperplane, we utilize parameters defined as:

$$\theta_i \cdot x = 0$$

This decomposition helps us visualize the separation between classes in the given feature space.

Softmax Function

Now we will transform the computed values into a probability distribution using the **softmax function**. The general format is given by:

$$P(y = k) = \frac{\exp(\theta_k \cdot x)}{\sum_{j=1}^{k} \exp(\theta_j \cdot x)}$$

The probabilities are normalized to sum to 1, allowing us to classify the input points based on their computed probabilities. E.g:

$$\begin{array}{lcll} \theta_1 x = 0.7 & & e^{0.7} & e^{0.7}/e^{0.7} + e^{-0.5} + e^{-0.5} + e^{-0.2} \approx 46\% \\ \theta_2 x = -0.5 & \text{convert} & e^{-0.5} & e^{-0.5}/e^{0.7} + e^{-0.5} + e^{-0.5} + e^{-0.2} \approx 14\% \\ \theta_3 x = -0.1 & \Longrightarrow & e^{-0.1} & \Longrightarrow \quad e^{-0.1}/e^{0.7} + e^{-0.5} + e^{-0.5} + e^{-0.2} \approx 21\% \\ \theta_4 x = -0.2 & & e^{-0.2} & e^{-0.2}/e^{0.7} + e^{-0.5} + e^{-0.5} + e^{-0.2} \approx 19\% \end{array}$$

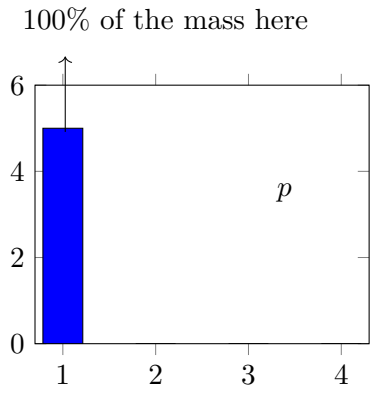
We have covered the fundamentals of multiclass classification, focusing on how to represent data, compute relevant values, and convert them into a probability distribution using the softmax function. Understanding these concepts is essential for implementing effective classification models in machine learning.

IV. Cross Entropy

In this section, we will discuss training models for multi-class classification. We will cover one-hot labels to represent true category distributions, how we estimate probabilities for each class during training, the concept of cross-entropy to quantify the difference between true and estimated distributions, and the role of the softmax function in converting raw outputs into interpretable probabilities.

- **Training Models:**

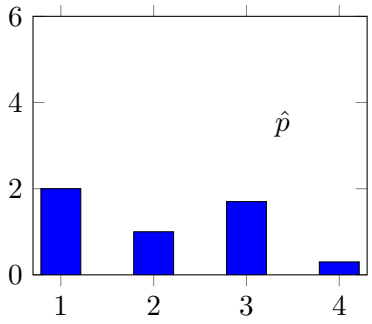
  When training models, we begin with one-hot labels indicating which class an instance belongs to. For example, if we are working with four classes, our true probability distribution can be represented as follows:



  In this diagram, p represents our true probability distribution where all the mass is on class one, indicating 100% confidence in that class.

- **Estimating Probabilities:**

  While training, our model will estimate probabilities for the classes, but it will often not be completely confident. This means we might have a distribution like the following:



In this second diagram, $\hat{p}$ represents our estimated probability distribution for the classes, capturing some uncertainty in the predictions.

Cross Entropy and Softmax

To train our models, we minimize the **cross-entropy loss** defined as:

$$C(p, \hat{p}) = -\sum_{y=1}^{k} p(y) \log(\hat{p}(y))$$

This loss function reflects the difference between the true probability distribution (p) and the estimated distribution ($\hat{p}$). By minimizing the cross-entropy, we are aligning our estimated probabilities with the true distribution.

When we have one-hot encoded p, only one term in the summation survives for the true class. If $p(y = k)$, we get:

$$C(p, \hat{p}) = -\log(\hat{p}(y_k))$$

This shows that minimizing the cross-entropy is equivalent to maximizing the likelihood of the true class under our model, which leads to effective learning.

Both minimizing cross-entropy and applying the softmax function yield the same results in the context of multiclass classification, specifically when dealing with categorical outcomes.

The process of training models involves estimating class probabilities based on one-hot labels and minimizing the cross-entropy loss. The softmax function plays an essential role in translating raw model outputs into meaningful probability distributions. Understanding these concepts is crucial for effective machine learning model training.

V. Summary

In this lecture on Multiclass Classification with Softmax Regression, we have covered the following key concepts:

- **Multiclass Softmax Regression:** This approach is used for handling multiple discrete outcomes and can be modeled using a multinomial distribution.
- **One-Hot Encoding:** A technique to represent categorical outcomes as binary vectors. Each class is represented by a unique vector containing a single '1' and the rest '0's, making it suitable for machine learning algorithms.
- **Hyperplanes:** In multiclass classification, hyperplanes are used to separate different classes in the feature space, providing a geometric interpretation of class boundaries.
- **Softmax Function:** This function transforms the output scores from the model into a probability distribution across the classes, ensuring that probabilities sum to 1.
- **Cross-Entropy Loss:** A crucial loss function for training models, measuring the discrepancy between the true distribution (one-hot encoded) and the estimated probabilities generated from the softmax function.
- **Probabilities Estimation:** The model estimates probabilities for each class, which may not always show complete confidence, especially in cases of uncertainty.
- **Connection of Concepts:** The interplay between one-hot encoding, softmax, and cross-entropy is essential for effective training of multiclass classification models in machine learning.

Module 4: Supervised Learning
Submodule: 4.6 Gaussian Discriminant Analysis
Lecture: Generative Learning Algorithms

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this lecture, we will discuss **generative learning algorithms**. We will define what it means by generative learning algorithms and cover two types:

- **Gaussian Discriminative Analysis (GDA)**
- **Naive Bayes**

II. Discriminative Learning Algorithms

Discriminative learning algorithms model the conditional relationship of y given x. Therefore, the main focus is:

$$p(y \mid x)$$

In the previous lectures, we modeled y as a function of x. For example, we can express this as:

$$y \mid x; \theta = \text{Exponential family}(\eta), \qquad \eta = \theta^T x.$$

where η is a linear function of x. This is why we refer to these as discriminative learning algorithms. For example, you can say this is a Gaussian distribution, with mean η. And that's just the standard linear regression.

III. Generative Learning Algorithms

Generative learning algorithms aim to model or parameterize the joint distribution $p(x, y)$. This is expressed mathematically as:

$$p(x, y) = p(x \mid y) \cdot p(y)$$

Here:

- x is the inputs
- y a label or some kind of class
- $p(y)$ is the distribution of the labels (also called a prior for the class or the label)
- $p(x \mid y)$ is the distribution of the input given the label

Note that x and y are not symmetric; typically, y represents the label (outcome), whereas x is the input features. For example, if y is the price of a house, then x could include features such as square footage, lot size, etc.

Here, $p(y)$ is sometimes referred to as the prior distribution for the label or class. This reflects what we believe about the distribution of classes before observing any data. In classification problems, this could be:

$$\begin{cases} y = 1 & \text{(positive sentiment)} \\ y = 0 & \text{(negative sentiment)} \end{cases}$$

This indicates the distribution over two labels in our dataset.

IV. Learning the Distributions

After modeling and parameterizing both distributions, we aim to learn them:

$$p(x \mid y) \quad \text{and} \quad p(y)$$

To solve the classification problem, we still want to compute $p(y \mid x)$. We will use **Bayes' rule**:

$$p(y \mid x) = \frac{p(x \mid y) \cdot p(y)}{p(x)}.$$

To understand $p(x)$, we can compute it using the law of total probability:

$$p(x) = \sum_{y'} p(x \mid y') \cdot p(y').$$

We denote y' as the variable from the summation. Thus,

$$p(y \mid x) = \frac{p(x \mid y) \cdot p(y)}{p(x)} = \frac{p(x \mid y) \cdot p(y)}{\sum_{y'} p(x \mid y') \cdot p(y')}.$$

Using Bayes' theorem:

- You will know $p(x \mid y)$ and $p(y)$.
- The denominator $p(x)$ can be computed as shown, although in many cases you may not need to compute it directly.

After knowing these quantities, $p(y \mid x)$ can be found using Bayes' rule.

While discriminative learning algorithms require only the conditional distribution $p(y \mid x)$ to be parameterized by some θ, generative algorithms require a full description of the joint distribution. If the distributions belong to the exponential family, there are several advantages.

V. Summary

- **Discriminative Learning:** focuses on modeling $p(y \mid x)$, the conditional probability of the label given the input.
- **Generative Learning:** focuses on modeling the joint distribution $p(x, y) = p(x \mid y) \cdot p(y)$.
- **Bayes' rule:** is applied to compute $p(y \mid x)$ from $p(x \mid y)$ and $p(y)$.
- **Gaussian Discriminant Analysis (GDA) and Naive Bayes:** are examples of generative models.
- **Generative models:** are beneficial when the distributions follow the exponential family due to their flexibility and richness in modeling data.

Module 4: Supervised Learning
Submodule: 4.6 Gaussian Discriminant Analysis
Lecture: Gaussian Discriminant Analysis (GDA)

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this lecture, we discuss two instantiations of the general concept:

- **Continuous features** x (GDA)
- **Discrete features** x (spam filtering)

We will primarily focus on the continuous case, which is referred to as **Gaussian Discriminant Analysis** (GDA). In the next lecture, we will cover an application to spam filtering.

II. Gaussian Discriminant Analysis (GDA)

Modeling Assumption

For GDA, we make the following modeling assumption:

$$p(x \mid y) \text{ follows a Gaussian distribution.}$$

This implies that x given y follows a Gaussian distribution with some mean and covariance. Specifically, we will work with a high-dimensional vector $x \in \mathbb{R}^d$ (for convenience we drop the $x_0 = 1$ convention for the bias term), where the Gaussian distribution can be represented as:

$$x \mid y \sim \mathcal{N}(\mu, \Sigma)$$

where μ is the mean vector and Σ is the covariance matrix.

Multivariate Gaussian Distribution Review

The multivariate normal distribution in d-dimensions, also called the **multivariate Gaussian distribution** $z \sim \mathcal{N}(\mu, \Sigma)$ where $z \in \mathbb{R}^d$ is parameterized by:

- Mean vector: $\mu \in \mathbb{R}^d$.
$$\mathbb{E}[z] = \mu$$
- Covariance matrix: $\Sigma \in \mathbb{R}^{d \times d}$,

$$\text{Cov}(z) \stackrel{\triangle}{=} \mathbb{E}[(z - \mathbb{E}[z])(z - \mathbb{E}.[z])^T] = \Sigma$$

The entries in the covariance matrix Σ_{ij} capture the correlation between the i^{th} and j^{th} dimensions. For example:

$$\Sigma_{ij} = \text{Cov}(z_i, z_j) = \mathbb{E}[(z - \mathbb{E}[z])(z - \mathbb{E}.[z])^T]$$

If z_1 and z_2 are correlated, their density contours will be ellipses rather than circular shapes, reflecting the direction of the correlation.

The probability density function of a multivariate Gaussian distribution is given by:

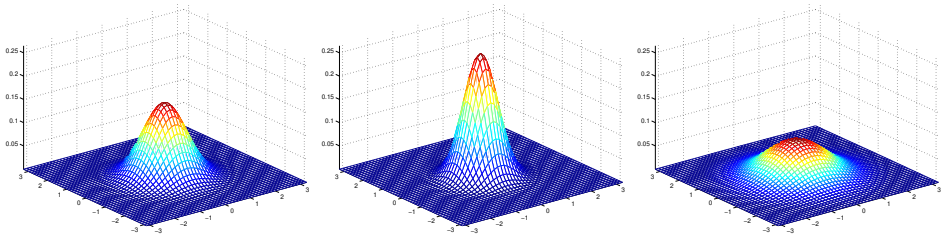
$$p(z) = \frac{1}{(2\pi)^{d/2}\sqrt{|\Sigma|}} e^{-\frac{1}{2}(z-\mu)^T \Sigma^{-1}(z-\mu)}$$

where $|\Sigma|$ is the determinant of the matrix Σ.

The probability density of the Gaussian distribution exhibits certain characteristics:

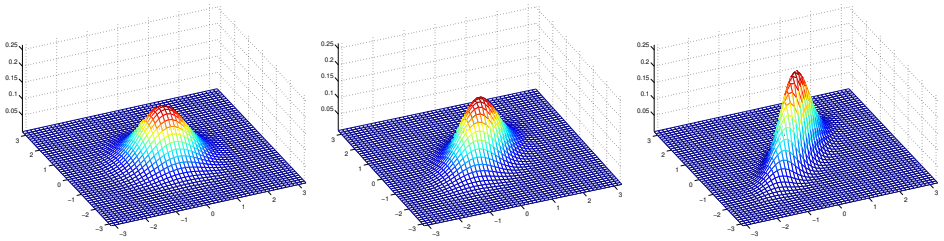
- If Σ is the identity matrix, there are no correlations between dimensions.
- Changes in Σ affect both the size and orientation of the probability density function.

The shapes of these distributions can be visualized in 2D, and the effectiveness of the covariance matrix impacts the density function's characteristics. Here are some examples of what the density of a Gaussian distribution looks like:



- The left-most figure: shows a Gaussian with mean zero (that is, the 2x1 zero-vector) and covariance matrix $\Sigma = I$ (the 2x2 identity matrix). A Gaussian with zero mean and identity covariance is also called the **standard normal distribution**.
- The middle figure: shows the density of a Gaussian with zero mean and $\Sigma = 0.6I$.
- The right-most figure: shows one with $\Sigma = 2I$. We see that as Σ becomes larger, the Gaussian becomes more spread-out, and as it becomes smaller, the distribution becomes more compressed.

Some more examples.



The figures above show Gaussians with mean 0 and covariance matrices respectively:

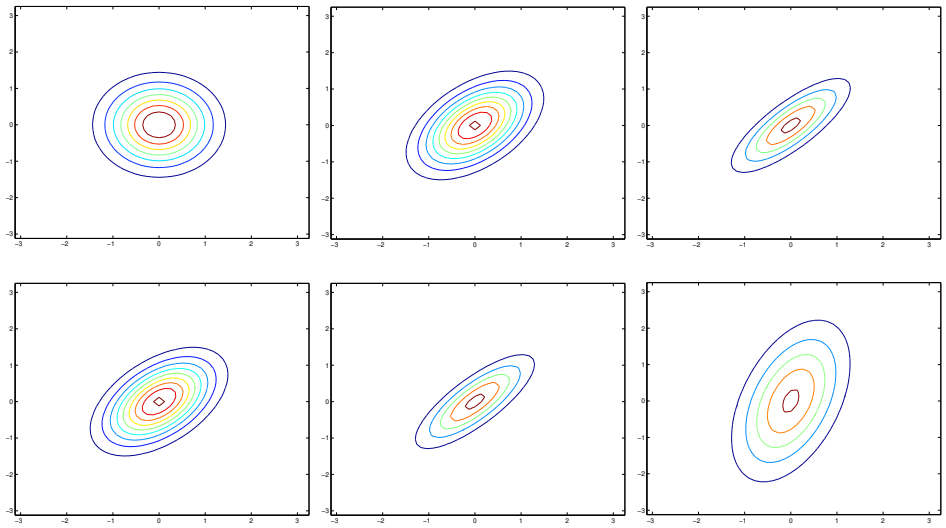
$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$

The leftmost figure shows the familiar standard normal distribution, and we see that as we increase the off-diagonal entry in Σ, the density becomes more compressed towards the 45° line (given by $x_1 = x_2$). We can see this more clearly when we look at the contours of the same three densities:

Here's one last set of examples generated by varying Σ:

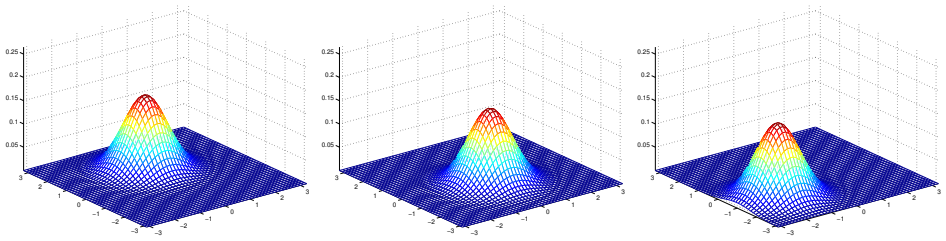
The plots above used, respectively,

$$\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$



From the leftmost and middle figures, we see that by decreasing the off-diagonal elements of the covariance matrix, the density now becomes "compressed" again, but in the opposite direction. Lastly, as we vary the parameters, more generally the contours will form ellipses (the rightmost figure showing an example).

As our last set of examples, fixing $\Sigma = I$, by varying μ, we can also move the mean of the density around.



The figures above were generated using $\Sigma = I$, and respectively

$$\mu = \begin{bmatrix} 1 \\ 0 \end{bmatrix}; \quad \mu = \begin{bmatrix} -0.5 \\ 0 \end{bmatrix}; \quad \mu = \begin{bmatrix} -1 \\ -1.5 \end{bmatrix}.$$

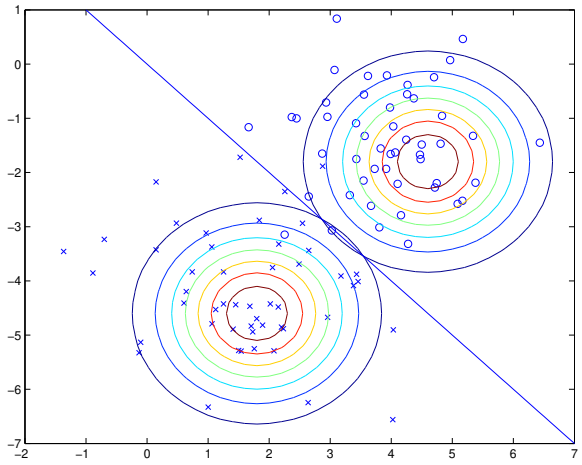
III. Model of GDA

In GDA, we assume discrete labels y. For the classification task, we focus on two labels: 0 and 1.

$$y \in \{0, 1\}$$

Let us consider a case of cancer classification where x_1 and x_2 are measurements. We want to classify cases as benign (0) or malignant (1).

The figure below shows the training set along with the contours of the two Gaussian distributions that have been fitted to the data in each of the two classes.



The two Gaussians have the same shape and orientation since they share a common covariance matrix, Σ. However, the Gaussians have different means, μ_0 and μ_1, for each class.

Additionally, the figure displays a straight line representing the decision boundary at which $p(y = 1 \mid x) = 0.5$. On one side of the boundary, we predict $y = 1$ as the most likely outcome. On the other side, we predict $y = 0$ as the most likely outcome. This decision boundary is crucial for classification tasks, where the goal is to assign class labels based on the input features.

Parameterizing x given y

We define:

$$\begin{aligned} x \mid (y = 0) &\sim \mathcal{N}(\mu_0, \Sigma) \\ x \mid (y = 1) &\sim \mathcal{N}(\mu_1, \Sigma) \end{aligned}$$

Here μ_0 and μ_1 are the mean vectors for each class $\mu_0, \mu_1 \in \mathbb{R}^d$, while $\Sigma \in \mathbb{R}^{d \times d}$ is the common covariance matrix across both classes.

The prior distribution over the classes $p(y)$ is modeled using parameters:

$$\begin{aligned} \phi &= p(y = 1) \\ 1 - \phi &= p(y = 0) \end{aligned}$$

Thus, y can be treated as a **Bernoulli distribution** with parameter ϕ.

$$y \sim \text{Bernoulli}(\phi)$$

In summary, the parameters we need to learn are:

$$\mu_0, \mu_1, \Sigma \text{ and } \phi$$

Our goal is to learn these parameters so that we can compute $p(y \mid x)$ using the relationships established.

IV. Summary

In this lecture, we covered the core concepts of **Gaussian Discriminant Analysis (GDA)** with a focus on modeling continuous features using Gaussian distributions. Below are the key points:

- **Modeling Assumptions:** In GDA, we assume that the data for each class follows a Gaussian distribution. Specifically, the features x conditioned on the class y are modeled as multivariate Gaussian distributions with means μ_0 and μ_1, and a shared covariance matrix Σ.
- **Multivariate Gaussian Distribution:** The Gaussian distribution in d dimensions is parameterized by a mean vector $\mu \in \mathbb{R}^d$ and a covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$. The covariance matrix captures the relationship between the dimensions of the data and affects the shape of the distribution (e.g., how spread out or compressed the contours are).
- **Covariance Matrix Impact:** By adjusting the covariance matrix Σ, we can modify the shape of the Gaussian probability density. A larger value of Σ leads to more spread-out contours, while a smaller Σ results in a more compressed distribution. The off-diagonal elements of Σ influence the correlation between dimensions which is reflected in the orientation of the contours.
- **Decision Boundary:** The decision boundary in GDA is defined as the point where the posterior probability $p(y = 1 \mid x) = 0.5$. On one side of the boundary, we predict class $y = 1$ and on the other side, we predict $y = 0$. The boundary is determined based on the Gaussian distributions of the two classes and the prior probability of the classes.
- **Learning Parameters:** The goal in GDA is to learn the parameters of the Gaussian distributions, including the mean vectors μ_0 and μ_1, the covariance matrix Σ, and the prior probability ϕ of class $y = 1$. Once these parameters are learned, we can use Bayes' rule to compute the posterior probability $p(y \mid x)$ and classify new data points.

- **Gaussian Distributions in Classification:** GDA is particularly useful in classification tasks, such as the cancer classification example, where we aim to distinguish between two classes (e.g., benign vs. malignant) based on input features.

Overall, GDA is a powerful method that allows us to model the distribution of features for each class and use this information to classify new data based on learned parameters.

Module 4: Supervised Learning
Submodule: 4.6 Gaussian Discriminant Analysis
Lecture: Maximum Likelihood Estimation (MLE)

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this lecture, we discuss how to learn parameters from data using the principle of **Maximum Likelihood** (ML).

II. Likelihood Function

The likelihood function, denoted as $L(\phi, \mu_0, \mu_1, \Sigma)$, describes how likely it is to observe the given data based on certain parameters. This function is central to the **Maximum Likelihood Estimation** (MLE) method.

$$L(\phi, \mu_0, \mu_1, \Sigma) = P\left(\left(x^{(1)}, y^{(1)}\right), \cdots, \left(x^{(n)}, y^{(n)}\right); \phi, \mu_0, \mu_1, \Sigma\right).$$

This equation states that the likelihood L is equal to the probability of observing all data points $(x^{(i)}, y^{(i)})$ given the parameters $\phi, \mu_0, \mu_1, \Sigma$. Here, i represents the i^{th} example from a dataset of n examples.

$$L(\phi, \mu_0, \mu_1, \Sigma) = \prod_{i=1}^{n} P\left(\left(x^{(i)}, y^{(i)}\right); \phi, \mu_0, \mu_1, \Sigma\right).$$

Since we assume the data are drawn independently from each other, we can factor the joint probability into a product of individual probabilities. This means that the overall likelihood is the product of the likelihoods of each example.

$$L(\phi, \mu_0, \mu_1, \Sigma) = \prod_{i=1}^{n} P\left(x^{(i)} \mid y^{(i)}; \phi, \mu_0, \mu_1, \Sigma\right) P\left(y^{(i)}; \phi, \mu_0, \mu_1, \Sigma\right).$$

Using the chain rule of probability, we can expand the individual likelihood for $(x^{(i)}, y^{(i)})$ into the conditional probability of $x^{(i)}$ given $y^{(i)}$ multiplied by the probability of $y^{(i)}$ itself. The likelihood is thus expressed with respect to $y^{(i)}$ and $x^{(i)}$.

$$L(\phi, \mu_0, \mu_1, \Sigma) = \prod_{i=1}^{n} P\left(x^{(i)} \mid y^{(i)}; \mu_0, \mu_1, \Sigma\right) P\left(y^{(i)}; \phi\right).$$

Here we can simplify further by recognizing that $y^{(i)}$ only depends on the parameter ϕ and not $\mu_0, \mu_1 \text{or} \Sigma$. Therefore, the probability of $y^{(i)}$ is written solely as $P\left(y^{(i)}; \phi\right)$, while the conditional probability of $x^{(i)}$ given $y^{(i)}$ now depends on μ_0, μ_1, Σ alone.

Finally, in Maximum Likelihood Estimation, we seek to maximize the likelihood function:

$$\max_{\phi, \mu_0, \mu_1, \Sigma} L(\phi, \mu_0, \mu_1, \Sigma).$$

This leads us to find the most probable parameters $\phi, \mu_0, \mu_1, \Sigma$ that explain the observed data.

The independence assumption is common in machine learning because it often reflects the nature of data collection. However, dependencies can exist in certain scenarios (e.g., reinforcement learning).

III. Distinction Between Generative and Discriminative Learning Algorithms

In this section, we briefly highlight the key differences between generative and discriminative learning. Generative learning aims to model how the data is generated by learning the joint distribution of inputs x and outputs y. In contrast, discriminative learning focuses on modeling the decision boundary between classes by estimating the conditional probability of the output y given the input x. Understanding these differences is essential for choosing the appropriate model based on the specific problem we are addressing. Now, lets delve into generative learning first.

Generative Learning

- **Maximum Likelihood**:
  - In generative algorithms, we maximize the joint probability of the input x and output y.
  - Specifically, the aim is to maximize the joint probability density of the occurrences of both x and y.
- **Joint Probability**:

$$\prod_{i=1}^{n} P\left(x^{(i)} \mid y^{(i)}; \mu_0, \mu_1, \Sigma\right) P\left(y^{(i)}; \phi\right).$$

  - This equation illustrates that we model the joint probability by multiplying the likelihood of $x^{(i)}$ given $y^{(i)}$ and the prior probability of $y^{(i)}$.

- **Assumptions**:
  - Generative approaches impose assumptions on both x and y. This means that you model the entire data structure, including how both variables interact.

Discriminative Learning

- **Conditional Likelihood**:
  - In contrast, discriminative algorithms focus on maximizing the conditional likelihood, or simply the likelihood of y given x.
  - This is often referred to as modeling the output y directly conditioned on the input x and parameters θ.
- **Modeling Focus**:

  - Discriminative models do not concern themselves with how x is generated. Instead, they target how y is derived from x with the assumption that x is a deterministic quantity that is observed.

- **Assumptions**:
  - The discriminative approach imposes fewer assumptions since it only models y given x. This limitation may lead to a more direct focus on the decision boundary between classes without needing to consider the distribution of x.

- **Conditional Probability**:

$$P\left(y^{(1)}, \cdots, y^{(n)} \mid x^{(1)}, \cdots, x^{(n)}; \theta\right) = \prod_{i=1}^{n} P\left(y^{(i)} \mid x^{(i)}; \theta\right).$$

  - This equation shows that the conditional likelihood is expressed as the product of the likelihoods of each $y^{(i)}$ given its corresponding $x^{(i)}$ under the parameters θ.

Summary of Differences

- **Focus**: Generative models deal with the full joint distribution of both x and y, while discriminative models focus solely on the boundary between different classes of y, given x.

- **Assumptions**: Generative models assume both input and output distributions; discriminative models assume only the output distribution conditioned on the input.

- **Trade-offs**:
  - With generative models, theres a risk of overfitting because they strive to learn more about the full data.
  - With discriminative models the challenge lies in capturing enough information about y based on x alone.

Practical Considerations

- Deciding to use a generative or discriminative approach often depends on domain-specific knowledge and the structure of the data. If underlying assumptions about data are accurate (e.g., assuming Gaussian distributions), a generative model may generalize well.

- Conversely, if assumptions do not hold, it could lead to errors, emphasizing the delicate balance in model selection.

IV. Maximizing the Likelihood

In this section, we discuss how to maximize the likelihood function $L(\phi, \mu_0, \mu_1, \Sigma)$. This is crucial for empirically estimating the parameters of our model.

The **likelihood function** is expressed as:

$$\arg \max_{\phi, \mu_0, \mu_1, \Sigma} L(\phi, \mu_0, \mu_1, \Sigma).$$

To simplify computation, we often use the **log-likelihood**:

$$\begin{aligned} \ell(\phi, \mu_0, \mu_1, \Sigma) &= \arg\max \log L(\phi, \mu_0, \mu_1, \Sigma) \\ &= \arg\max \sum_{i=1}^{n} \log p\left(x^{(i)} | y^{(i)}; \mu_0, \mu_1, \Sigma\right) + \log p\left(y^{(i)}; \phi\right) \end{aligned}$$

Taking the logarithm of the likelihood function transforms the product of probabilities into a sum, which is mathematically easier to handle. This transformation is important because it simplifies computations and makes the resulting values more manageable, especially when dealing with very small or very large numbers.

To find the parameters that maximize the log-likelihood, we use the fact from calculus that the **gradient** of a function is zero at its maximum:

$$\nabla \ell(\phi, \mu_0, \mu_1, \Sigma) = 0.$$

This implies solving the following system of equations:

$$\frac{\partial \ell}{\partial \phi} = 0, \quad \frac{\partial \ell}{\partial \mu_0} = 0, \quad \frac{\partial \ell}{\partial \mu_1} = 0, \quad \frac{\partial \ell}{\partial \Sigma} = 0.$$

To solve these equations, we need to set the partial derivatives with respect to each parameter to zero. (Note that in this course, the derivative of any parameter will have the same shape as the parameter. As such, $\partial \ell / \partial \phi \in \mathbb{R}$ and $\partial \ell / \partial \mu_0, \partial \ell / \partial \mu_1, \partial \ell / \partial \Sigma \in \mathbb{R}^d$.) This yields four equations that must be solved simultaneously. This analytical approach is often preferred over numerical optimization because it allows for direct computation of the maximizer without iterative algorithms.

The solutions for the parameters μ_0, μ_1, ϕ and Σ obtained from these equations will provide meaningful insights into the model. Understanding these parameters will help you interpret the underlying data distribution and how well the model fits the data.

Now that we have established the framework for maximizing the likelihood, we will proceed to derive specific solutions for the parameters and interpret their significance.

V. Maximum Likelihood Estimation (MLE) Solutions

In this section, we provide solutions for the parameters involved in our model based on the Maximum Likelihood Estimation (MLE) approach.

Let us define the following notations:

$$\begin{aligned} U_0 &= \{i : y^{(i)} = 0\} \quad \text{(indices for the negative examples)} \\ U_1 &= \{i : y^{(i)} = 1\} \quad \text{(indices for the positive examples)} \end{aligned}$$

The total number of examples is defined as:

$$n = |U_0| + |U_1|$$

where $|U_1|$ and $|U_0|$ denote the cardinality (size) of the sets of positive and negative examples, respectively.

The indicator function is defined as:

$$\mathbb{1}(E) = \begin{cases} 1 & \text{if } E \text{ happens,} \\ 0 & \text{otherwise.} \end{cases}$$

Estimating Parameter ϕ

The parameter ϕ, representing the fraction of positive examples, can be computed as:

$$\phi = \frac{|U_1|}{n}$$

This is the maximum likelihood estimate for the probability that $y = 1$.

Consequently, the fraction of negative examples is:

$$1 - \phi = \frac{|U_0|}{n}$$

Alternatively, we can express ϕ using the indicator function:

$$\phi = \frac{1}{n} \sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\}.$$

Estimating Parameters μ_0 and μ_1

Next, we estimate the means for the negative and positive examples as follows:

$$\mu_0 = \frac{1}{|U_0|} \sum_{i \in U_0} x^{(i)} = \frac{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 0\} \cdot x^{(i)}}{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 0\}}$$

This gives us the average of our negative examples.

Similarly, for the positive examples:

$$\mu_1 = \frac{1}{|U_1|} \sum_{i \in U_1} x^{(i)} = \frac{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\} \cdot x^{(i)}}{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\}}$$

This gives us the average of our positive examples.

Estimating Covariance Matrix Σ

The covariance matrix Σ can be estimated as:

$$\begin{aligned}
\Sigma &= \frac{1}{n}\left(\sum_{i\in U_0}(x^{(i)}-\mu_0)(x^{(i)}-\mu_0)^T + \sum_{i\in U_1}(x^{(i)}-\mu_1)(x^{(i)}-\mu_1)^T\right) \\
&= \frac{1}{n}\sum_{i=1}^{n}\left(x^{(i)}-\mu_{y^{(i)}}\right)\left(x^{(i)}-\mu_{y^{(i)}}\right)^T \\
&= \frac{1}{n}\left(\sum_{i\in U_0}\left(x^{(i)}-\mu_0\right)\left(x^{(i)}-\mu_0\right)^T + \sum_{i\in U_1}\left(x^{(i)}-\mu_1\right)\left(x^{(i)}-\mu_1\right)^T\right).
\end{aligned}$$

This formula provides the maximum likelihood estimate for the covariance matrix, accounting for both negative and positive examples.

At this point, we derived the MLE for all parameters ϕ, μ_0, μ_1, and Σ. These parameters help us understand the distribution of our data.

VI. Summary

In this lecture, we covered essential concepts and methods related to **Maximum Likelihood Estimation** (MLE) within the context of Gaussian Discriminant Analysis. Here are the key takeaways:

- **Likelihood Function:** The likelihood function $L(\phi, \mu_0, \mu_1, \Sigma)$ measures the probability of observing the data given certain parameters. Our goal is to maximize this function to find the best-fit parameters for our model.
- **Log-Likelihood:** We often work with the log-likelihood $\ell(\phi, \mu_0, \mu_1, \Sigma)$, which simplifies the computation by transforming products into sums.
- **Parameter Estimation:**
  - The parameter ϕ represents the fraction of positive examples in the dataset.
  - The parameters μ_0 and μ_1 denote the means of the negative and positive classes, respectively.
  - The covariance matrix Σ captures the variability within the classes.
- **Generative vs. Discriminative Models:** Understanding the distinction between generative models (which model the joint distribution of x and y) and discriminative models (which model the conditional probability of y given x) is crucial in selecting the appropriate learning approach.

- **Practical Implications:** The choice between generative and discriminative models should be informed by the nature of the data and the assumptions you can reasonably make, which can greatly impact model performance and the validity of predictions.

These concepts form a foundational understanding of how to approach parameter estimation in Gaussian Discriminant Analysis using Maximum Likelihood Estimation. In the next lectures, we will discuss how to utilize these parameters for making predictions on new examples based on the learned model.

Module 4: Supervised Learning
Submodule: 4.6 Gaussian Discriminant Analysis
Lecture: Prediction with GDA

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this lecture, we discuss the concept of **prediction**. Our goal is to output a value y specifically to understand whether a given example x is benign cancer or not. (I.e., given x, output $y \in \{0, 1\}$.) The final goal is to determine whether the prediction for y indicates benign (0) or malignant cancer (1).

II. Maximizing Probability

To achieve this, we can state that we will output the most likely value of y. This is given by:

$$\arg\max p(y|x; \phi, \mu_0, \mu_1, \Sigma).$$

It is important to note that the parameters ϕ, μ_0, μ_1 and Σ are not arbitrary; they are solutions of MLE equations.

III. Maximizing Over Two Classes

In our case, we are maximizing over two possible choices for y: specifically whether the examples is more likely to be benign (0) or malignant (1). This leads us to compare two scalar values:

$$\arg\max \Big\{ p(y=1|x;\phi,\mu_0,\mu_1,\Sigma),\ p(y=0|x;\phi,\mu_0,\mu_1,\Sigma) \Big\}.$$

For simplicity, suppose:

$$\begin{aligned} A &= p(y=1|x;\phi,\mu_0,\mu_1,\Sigma). \\ B &= p(y=0|x;\phi,\mu_0,\mu_1,\Sigma). \end{aligned}$$

Thus, $A+B=1$. Since we are interested in determining which of A or B is larger, we can redefine our problem. We are essentially checking if $A > 0.5$ or $A < 0.5$.

$$\max\{A,B\} = \begin{cases} A & \text{if } A \geq 0.5 \geq B, \\ B & \text{if } A \leq 0.5 \leq B. \end{cases}$$

So effectively we can summarize:

$$\max\{A,B\} = \begin{cases} 1 & \text{if } p(y=1|x;\phi,\mu_0,\mu_1,\Sigma) \geq 0.5, \\ 0 & \text{if } p(y=1|x;\phi,\mu_0,\mu_1,\Sigma) \leq 0.5. \end{cases}$$

IV. Decision Boundary

In this section, we explore the final decision and the boundary between two cases in terms of probability. The line that defines the decision boundary is characterized by the following condition:

$$\{x : p(y=1|x) = 0.5\}$$

This means we are looking for a family of values x such that the probability of y given x equals exactly 0.5. This defines our decision boundary. On one side of this boundary, $p(y=1|x)$ is more likely. Conversely, on the other side, $p(y=0|x)$ is more likely. (It is essential to note that the boundary resulting from arbitrary groupings or random outputs will not frequently result in points lying precisely on the boundary.)

In the context of GDA, using Bayes' Rule we can reformulate our probability of interest:

$$p(y=1|x) = \frac{p(x|y=1)p(y=1)}{p(x)}$$

Applying this to $p(y = 1|x; \phi, \mu_0, \mu_1, \Sigma)$ we get:

$$p(y = 1|x; \phi, \mu_0, \mu_1, \Sigma) = \frac{p(x \mid y = 1; \mu_1, \Sigma) \cdot p(y = 1; \phi)}{p(x; \phi, \mu_0, \mu_1, \Sigma)}.$$

For Gaussian Discriminant Analysis, if we view the quantity $p(y = 1|x; \phi, \mu_0, \mu_1, \Sigma)$ as a function of x, we find that it can be expressed in the form:

$$p(y = 1|x; \phi, \mu_0, \mu_1, \Sigma) = \frac{1}{1 + \exp(-(\theta^T x + \theta_0))}.$$

where $\theta_0 \in \mathbb{R}$ and $\theta \in \mathbb{R}^d$ is a function of the parameters $\phi, \mu_0, \mu_1, \Sigma$.

To find the decision boundary, we set the probability equal to 0.5:

$$\frac{1}{1 + \exp(-(\theta^T x + \theta_0))} = 0.5.$$

This simplifies to:

$$\exp(-(\theta^T x + \theta_0)) = 1.$$

Taking the natural logarithm gives:

$$-\theta^T x - \theta_0 = 0.$$

Which leads to:

$$\theta^T x + \theta_0 = 0.$$

The implication of this decision boundary is:

$$p(y = 1|x) > 0.5 \iff \theta^T x + \theta_0 > 0$$

This indicates that the decision boundary is a linear function of x, which separates the predicted classes effectively.

In summary, we determined that the decision boundary derived from GDA is linear and separates the two classes effectively, allowing us to classify cases of cancer as benign or malignant based on the features represented by x.

V. Summary

In this lecture, we covered essential concepts related to **Gaussian Discriminant Analysis** (GDA) and its application in prediction. Here are the key takeaways:

- **Objective of Prediction:** The primary goal is to classify a case as either benign cancer (0) or malignant cancer (1) based on a set of features represented by x.

- **Maximizing Probability:** We define the most likely value of y using:

$$\arg\max p(y|x;\phi,\mu_0,\mu_1,\Sigma).$$

- **Two Classes Comparison:** We compare probabilities for two classes, $A = p(y = 1|x)$ and $B = p(y = 0|x)$, leading to a decision based on whether $A > 0.5$ or $B > 0.5$.

- **Decision Boundary:** The line that separates the two classes is defined by the condition $\{x : p(y = 1|x) = 0.5\}$ and can be expressed as:

$$\theta^T x + \theta_0 = 0.$$

- **Bayes' Rule Application:** We utilize Bayes' Rule to derive the probability $p(y = 1|x)$, which leads us to the logistic function:

$$p(y = 1|x;\phi,\mu_0,\mu_1,\Sigma) = \frac{1}{1+\exp(-(\theta^T x+\theta_0))}.$$

- **Linear Decision Boundary:** Finally, we conclude that the decision boundary is a linear function of x, which effectively separates benign from malignant cases.

- **Practical Implications:** Understanding this model equips us with the tools to classify cases in practical scenarios, emphasizing the importance of correctly estimating the parameters involved.

Module 4: Supervised Learning
Submodule: 4.6 Gaussian Discriminant Analysis
Lecture: Discriminative vs Generative Learning Algorithms

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

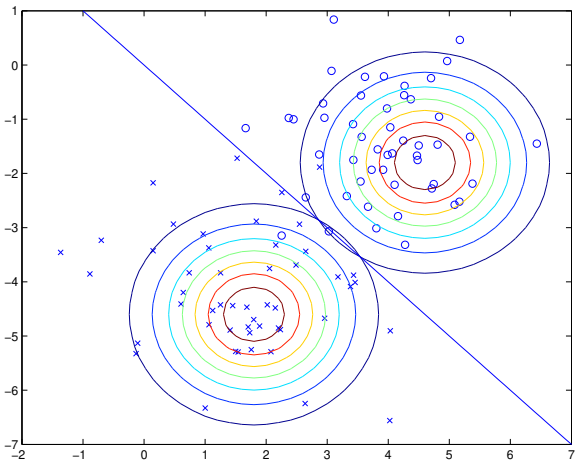
In this lecture, we discuss the Gaussian Discriminant Analysis (GDA) reviewed from our previous session and explore a new case concerning spam filtering, focusing on a discrete, rather than continuous, random variable x.

II. Review of Gaussian Discriminant Analysis

In GDA, we model the conditional probability $p(x|y)$ and the prior probability $p(y)$. Under the framework of GDA:

$$\begin{aligned} x \mid (y=0) &\sim \mathcal{N}(\mu_0, \Sigma). \\ x \mid (y=1) &\sim \mathcal{N}(\mu_1, \Sigma). \end{aligned}$$

Also, $p(y=1) = \phi$. With graphical representation could depict the clustering of positive and negative samples.



III. Parameter Estimation

To estimate the parameters ϕ, μ_0, μ_1 and Σ, we employ the Maximum Likelihood Estimation (MLE) method. The objective is to maximize the log-likelihood function:

$$\arg \max_{\phi,\mu_0,\mu_1,\Sigma} \sum_{i=1}^{n} \log p\left(x^{(i)}|y^{(i)};\mu_0,\mu_1,\Sigma\right) + \log p\left(y^{(i)};\phi\right)$$

The derived parameters from the MLE include:

- Probability of positive class:

$$\phi = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\}$$

- Means:

$$\mu_0 = \frac{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 0\} \cdot x^{(i)}}{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 0\}}$$

$$\mu_1 = \frac{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\} \cdot x^{(i)}}{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\}}$$

- Covariance:

$$\Sigma = \frac{1}{n}\left(\sum_{i\in U_0}\left(x^{(i)}-\mu_0\right)\left(x^{(i)}-\mu_0\right)^T + \sum_{i\in U_1}\left(x^{(i)}-\mu_1\right)\left(x^{(i)}-\mu_1\right)^T\right)$$

After learning the parameters, we make predictions for a new example x. The task is to compute the most likely value of y given x:

$$\arg \max_{y \in \{0,1\}} p(y|x; \phi, \mu_0, \mu_1, \Sigma)$$

The decision boundary is characterized by the equality:

$$\{x : p(y = 1|x) = p(y = 0|x) = 0.5\}$$

This results in the linear decision boundary:

$$\{x : \theta^T x + \theta_0 = 0\}$$

where θ and θ_0 are derived as functions of ϕ, μ_0, μ_1 and Σ.

IV. Comparison with Logistic Regression

The primary distinctions between GDA and logistic regression lie in:

- GDA makes strong assumptions about the underlying distributions of $p(x|y)$ (Gaussian).

$$\begin{aligned} x \mid (y = 0) &\sim \mathcal{N}(\mu_0, \Sigma). \\ x \mid (y = 1) &\sim \mathcal{N}(\mu_1, \Sigma). \\ y &\sim \text{Bernoulli}(\phi). \end{aligned}$$

- Logistic regression directly models $p(y|x)$ without assuming distributions for x. In logistic regression, the model takes the form:

$$p(y = 1|x) = \frac{1}{1 + e^{-(\theta^T x + \theta_0)}}$$

  Thus, the differences in model assumptions lead to potential variances in the derived parameters θ and θ_0.

When determining which model to prefer between Gaussian Discriminant Analysis (GDA) and Logistic Regression, several factors come into play:

- **Decision Boundaries and Model Assumptions:** GDA and logistic regression will, in general, produce different decision boundaries when trained on the same dataset. It is important to consider:
  - If the conditional distribution $p(x|y)$ is multivariate Gaussian (with shared covariance Σ), then $p(y|x)$ necessarily follows a logistic function.

  - Conversely, $p(y|x)$ being a logistic function does not imply that $p(x|y)$ is multivariate Gaussian. This demonstrates that GDA makes stronger modeling assumptions about the data than logistic regression.

- **Asymptotic Efficiency of GDA:** GDA has certain advantages under specific conditions:
  - When the modeling assumptions of GDA are correct, particularly when $p(x|y)$ is indeed Gaussian (with shared Σ), GDA is considered **asymptotically efficient**.
  - This means that with large training sets ($n \to \infty$), GDA will outperform any other algorithm in accurately estimating $p(y|x)$.
  - GDA is generally expected to provide better performance even with smaller training set sizes if the Gaussian assumption holds.

- **Robustness of Logistic Regression:** On the other hand, logistic regression has its unique strengths:
  - Logistic regression makes significantly weaker assumptions about the data distributions, making it more robust to incorrect modeling assumptions.
  - There exist various distributions for $p(x|y)$ that could result in $p(y|x)$ taking the form of a logistic function. For example, if, $x \in \mathbb{N}$ (i.e., x is an integer):

    $$x|y = 0 \sim \text{Poisson}(\lambda_0),$$
    $$x|y = 1 \sim \text{Poisson}(\lambda_1),$$

    then $p(y|x)$ will still be logistic.

  In summary:

  - GDA:
    * Stronger modeling assumptions.
    * More data efficient: requires less training data to achieve good performance when assumptions are correct.
  - Logistic Regression:
    * Weaker assumptions, making it significantly more robust to modeling deviations.
    * Generally performs better than GDA when the data distribution is non-Gaussian, especially as dataset size increases.

  Due to its stronger robustness and lower sensitivity to violations of the underlying assumptions, logistic regression is often preferred in practice over GDA.

V. Summary

- GDA provides a framework for modeling when the underlying assumptions (Gaussian distributions) are valid, and can yield better estimates with sufficient data.
- The decision boundary derived from GDA can be expressed linearly, reflecting its nature as a generative model.
- Logistic regression offers greater flexibility and robustness, making fewer assumptions about the data distribution and often leading to better performance in varied scenarios.
- The choice between GDA and logistic regression should consider the nature of the data, the validity of assumptions about the underlying distributions, and the amount of available data.

Module 4: Supervised Learning
Submodule: 4.7 Naive Baye
Lecture: Naive Bayes Spam Classification

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this section, we introduce the problem of **spam classification**, which involves determining whether a piece of text (an email) is spam or not. We outline the goal of developing a generative learning algorithm to tackle this classification problem. And we are going to have discrete x.

II. Feature Representation for Spam Classification

We need to represent the text in a numerical format, as computers process numerical data. The aim is to convert a piece of text into a feature vector x of dimension d, where d represents the size of the vocabulary.

$$\text{text} \longrightarrow x \in \mathbb{R}^d$$

In our approach to constructing a spam filter, we will begin by specifying the features x_j used to represent an email.

We will represent each email using a feature vector whose length is equal to the number of words in our dictionary. We list all words from our vocabulary (10,000 words) in alphabetical order. Specifically, if an email contains the j-th word from the dictionary, we will set $x_j = 1$; otherwise, we will set $x_j = 0$.

For instance, consider the following feature vector:

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

This vector x is used to represent an email containing the words "a" and "buy," but not "aardvark," "aardwolf" or "zygmurgy" (which is a branch of applied chemistry that deals with the science and practice of fermentation... but you already knew that). The set of words that is encoded into the feature vector is referred to as the **vocabulary**. Consequently, the dimension of the vector x is equal to the size of the vocabulary (i.e., $d = 10,000$).

So if we have a sentence like "I buy a book", we create a vector $x \in \{0,1\}^d$ where:

$$x_i = \begin{cases} 1 & \text{if i}^{th}\text{-word appears in the email} \\ 0 & \text{if i}^{th}\text{-word does not appear in the email} \end{cases}$$

We label each entry corresponding to a word in the vocabulary, leading to a representation that captures the presence or absence of words regardless of order or frequency. This is simplistic because we don't capture the order of the words or the frequency with which they appear but this representation will simplify the math.

Now that we have established our feature vector x, which is a binary vector taking values of 0 or 1, we need a distribution that can generate these binary vectors. In this context, we will utilize the Naive Bayes classification technique.

III. Generative Model

In this section, we outline how to build a generative model for x given y. Here, y indicates whether the email is spam (1) or not spam (0).

Naive Bayes Assumption

The estimation of the joint distribution is done through the following factorization:

$$P(x|y) = P(x_1|y) \cdots P(x_d|y)$$

To model $p(x|y)$, we assume that $x_1, x_2, \ldots, x_d$ are *conditionally independent* given y. This means that once we know y, we can independently draw each feature x_i.

"Conditional independence" is the **Naive Bayes (NB) assumption**, and the resulting algorithm is called a **Naive Bayes classifier**.

It is important to note that this independence assumption may not accurately reflect how people generate emails. As an example:

- When generating a spam email, one might first decide if the email is spam (determine y) and then randomly select which features to include in the email. This is likely not how actual spam emails are written.

However, despite the unrealistic nature of this assumption, it turns out that in many cases, such simplifying assumptions can lead to effective outcomes. For instance, utilizing this Naive Bayes model in practice can achieve over 90% accuracy in spam classification.

As of 2012, the effectiveness of this approach may have diminished due to the adversarial nature of spammers, who can modify their emails to evade detection algorithms. Nonetheless, this method was a reasonable approach around ten years ago.

Interestingly, even if the assumption of independence is not entirely correct, it may still yield useful results due to the partial correctness of the assumption. For our purposes, the focus is less on the validity of the assumptions and more on demonstrating the methodology of applying this probabilistic model effectively.

Another consideration is how the length of the text impacts representation. In our binary vector model, the length of the original text does not play a significant role. We can encode any length of text into a single vector of dimension d (the size of the vocabulary). This can be viewed as both an advantage and a disadvantage.

When deciding what constitutes the "window size" for our feature vector, it is generally practical to take the entire email as the unit of classification, as we are assessing whether the complete email is spam or not. Therefore, the classification will determine whether each emails qualifies as spam, rather than classifying individual sentences.

IV. Parameterization of the Naive Bayes Model

In the Naive Bayes classifier, we represent the joint probability of the features x given a class label y as follows:

$$P(x_1, \ldots, x_d \mid y) = P(x_1 \mid y) \cdots P(x_d \mid y)$$

To understand this better, we need to parameterize each of these probability distributions using some parameters. Given that each feature x_i can only take on two values, 0 or 1, we note that these probabilities are typically modeled using a Bernoulli distribution. Therefore, for each feature, we can describe its distribution with a single parameter.

Let $\phi_{j|y=1} = P(x_j = 1 \mid y = 1)$ denote the probability of feature x_j being 1 when y is equal to 1 (indicating that the email is spam). Consequently, we have:

$$P(x_j = 0 \mid y = 1) = 1 - \phi_{j|y=1}$$

Similarly, we define another parameter:

$$\phi_{j|y=0} = P(x_j = 1 \mid y = 0)$$

which gives us:

$$P(x_j = 0 \mid y = 0) = 1 - \phi_{j|y=0}$$

Additionally, we need to parameterize the probability of the class label itself:

$$\phi_y = P(y = 1)$$

Here, ϕ_y serves to define the distribution of the class label y.

As we proceed, we will leverage these parameters to calculate the overall probabilities for classifying emails as spam using Naive Bayes.

V. Maximizing the Likelihood in Naive Bayes

In this section, we discuss how to write the likelihood function and subsequently maximize it.

The likelihood L is defined as the probability of observing the data given the parameters:

$$L(\phi_y, \phi_{1|y=0}, \ldots, \phi_{d|y=0}, \phi_{1|y=1}, \ldots, \phi_{d|y=1}) = \prod_{i=1}^{n} p(x^{(i)}, y^{(i)}; \phi_y, \phi_{j|y}).$$

Note, $\phi_y, \phi_{j|y}$ is just a convenient way to represent *all* the parameters, which are $\phi_y, \phi_{1|y=0}, \ldots, \phi_{d|y=0}, \phi_{1|y=1}, \ldots, \phi_{d|y=1}$.

The likelihood can be factored into the product of the probabilities of each example because all examples are assumed to be independent:

$$= \prod_{i=1}^{n} p(x^{(i)} \mid y^{(i)}; \phi_y, \phi_{j|y}) \cdot p(y^{(i)}; \phi_y, \phi_{j|y})$$

This can be further simplified using the chain rule:

$$= \prod_{i=1}^{n} \left(p(y^{(i)}; \phi_y) \cdot \prod_{j=1}^{d} p(x_j^{(i)} \mid y^{(i)}; \phi_{j|y}) \right)$$

The next step is to maximize this likelihood. We know that maximizing L is equivalent to maximizing $\log L$:

$$\arg\max L = \arg\max \log L$$

Converting the likelihood to its logarithmic form yields:

$$\log L(\phi_y, \phi_{j|y}) = \ell(\phi_y, \phi_{j|y}) = \sum_{i=1}^{n} \log p(y^{(i)}; \phi_y) + \sum_{i=1}^{n} \sum_{j=1}^{d} \log p(x_j^{(i)} \mid y^{(i)}; \phi_{j|y})$$

To find the maximum, we compute the gradient of the log-likelihood and set it to zero:

$$\nabla \ell(\phi_i, \phi_{j|y}) = 0.$$

Thus, solving the maximization yields parameter estimates as follows:

$$\begin{aligned}
\phi_y &= \frac{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\}}{n} \\
\phi_{j|y=1} &= \frac{\sum_{i=1}^{n} \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = 1\}}{\sum_{i=1}^{n} 1\{y^{(i)} = 1\}} \\
\phi_{j|y=0} &= \frac{\sum_{i=1}^{n} \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = 0\}}{\sum_{i=1}^{n} 1\{y^{(i)} = 0\}}
\end{aligned}$$

where $\mathbb{1}$ is the indicator function that represents whether the sample corresponds to a positive example. Here:

- ϕ_y represents the fraction of positive examples.
- $\phi_{j|y=1}$ gives the probability of the j^{th} word being 1 given the label y is 1 (spam), which can be interpreted as the count of occurrences of the j^{th} word across all positive examples, normalized by the total number of positive examples.
- Similarly, $\phi_{j|y=0}$ is defined for negative examples.

This shows that our model's estimates update based on the frequencies of the words in either class, making the method straightforward and intuitive.

VI. Summary

- **Spam Classification:** The goal is to determine whether an email is spam or not using a generative learning algorithm.
- **Feature Representation:** Text is converted into binary feature vectors, where each element represents the presence of a word from a predefined vocabulary. If the word is present, it is set to 1; otherwise, it is set to 0.
- **Generative Model:** The Naive Bayes model relies on the Naive Bayes assumption of conditional independence, meaning that each feature is independent given the class label (spam or not spam).
- **Model Parameterization:** The probabilities of features given the class are parameterized using terms like $\phi_{j|y=1}$ and $\phi_{j|y=0}$.
- **Maximizing the Likelihood:** The likelihood function L is maximized, and the logarithm of the likelihood is utilized for easier computation. Parameter estimates are derived from the frequency of words in the respective classes.
- **Practical Importance:** The Naive Bayes classifier can achieve good accuracy in spam detection, despite its simplifying assumptions.

Module 4: Supervised Learning
Submodule: 4.7 Naive Baye
Lecture: Prediction and Laplace Smoothing

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this lecture, we explore key concepts in predictive modeling, focusing on how to compute the probability $P(y = 1|x)$ for classification. We will introduce **Laplace smoothing** as a solution to the issue of zero probabilities for unseen features. By the end, you will understand how to manage predictions effectively, even with limited data.

II. Probability Computation

We want to compute $P(y = 1|x)$ as per the standard methodology:

$$P(y = 1|x) = \frac{P(x|y = 1)P(y = 1)}{P(x)}$$

If $P(y = 1|x) > 0.5$, we classify the example as positive; if $P(y = 1|x) < 0.5$, we classify as negative.

III. The Zero-Probability Problem

In this section, we discuss a common issue encountered in Naive Bayes classifiers when certain features do not appear in the training dataset, illustrated using the example of the word "aardvark."

Suppose the word "aardvark" never appears in the training set. As a result, for every example i:

$$x_2^{(i)} = 0, \quad \forall i$$

This indicates that the second feature (word) does not show up in any training example. We estimate probabilities for the feature given different classes:

$$\phi_{2|y=1} = \frac{0}{\#\text{ of positive examples}} = 0 \tag{1}$$

$$\phi_{2|y=0} = \frac{0}{\#\text{ of negative examples}} = 0 \tag{2}$$

To compute the overall probability $P(x)$, we apply the following equation:

$$P(x) = P(x|y = 1)P(y = 1) + P(x|y = 0)P(y = 0)$$

This can be further broken down into:

$$P(x) = \prod_{j=1}^{d} p(x_j|y = 1)P(y = 1) + \prod_{j=1}^{d} p(x_j|y = 0)P(y = 0)$$

Given the previous estimations:

$$\begin{aligned} p(x_2 = 1|y = 1) &= \phi_{2|y=1} = 0 \\ p(x_2 = 1|y = 0) &= \phi_{2|y=0} = 0 \end{aligned}$$

Thus, we have:

$$P(x) = 0 \cdot P(y = 1) + 0 \cdot P(y = 0) = 0$$

We conclude that:

$$P(y = 1|x) = \frac{P(x|y = 1)P(y = 1)}{P(x)} = \frac{0}{0}$$

This leads to an indeterminate form $\frac{0}{0}$ when calculating $P(y = 1|x)$.

This situation is problematic because it indicates that our model cannot handle unseen features, leading to zero probabilities. It is possible to observe entirely new words in your test set.

To mitigate this, we use Laplace smoothing, which allows us to introduce a small probability for unseen features. This adjustment indicates that we should not completely rely on the training data. Laplace smoothing proposes that instead of counting zero occurrences, we adjust the counts by adding one (or a small constant) to all counts to ensure no probability is exactly zero.

IV. Laplace Smoothing

To avoid the issue of zero probabilities in our model, we apply **Laplace smoothing**, which allows for a small probability on unseen features. This is crucial for maintaining sensible class predictions.

Simple example:

To estimate the bias of a coin, we model the outcomes of flipping the coin using a Bernoulli distribution. Let Z represent a random variable that indicates the result of a coin flip, where:

- $Z = 1$ corresponds to heads,
- $Z = 0$ corresponds to tails.

This can be mathematically expressed as:

$$Z \sim \text{Bernoulli}(\phi)$$

where ϕ is the unknown probability of getting heads.

Suppose we perform n trials of flipping the coin. The outcomes of these trials can be represented as follows:

$$\begin{array}{lllll} Z^{(1)} & \text{tail} & = 0 & (1-\phi) \\ Z^{(2)} & \text{tail} & = 0 & (1-\phi) \\ & \vdots & & \\ Z^{(n)} & \text{heads} & = 1 & \phi \end{array}$$

Here, each $Z^{(i)}$ results in a head or tail depending on the respective probability ϕ or $1 - \phi$.

The likelihood function $L(\phi)$, which represents the probability of observing the data given the parameter ϕ, can be formulated as:

$$L(\phi) = (1 - \phi)(1 - \phi) \cdots \phi = (1 - \phi)^t \cdot \phi^h$$

where t is the number of tails and h is the number of heads.

To find the value of ϕ that maximizes the likelihood function, we can take the argument that maximizes $\log L(\phi)$:

$$\arg\max_{\phi} \log L(\phi) = \frac{h}{h + t}$$

This formula indicates that the maximum likelihood estimate of ϕ is simply the ratio of heads observed in the trials. The expression $\frac{h}{h+t}$ provides an empirical estimate of the bias of the coin based on observed data, which is intuitive as it reflects the observed proportion of heads in the experiment.

Another example

Let us consider an example where we flip a coin three times and observe the results:

$$\begin{array}{lllll} Z^{(1)} & \text{tail} & = 0 & (1 - \phi) \\ Z^{(2)} & \text{tail} & = 0 & (1 - \phi) \\ Z^{(3)} & \text{tail} & = 0 & (1 - \phi) \end{array}$$

The standard estimate for ϕ would be:

$$\phi = \frac{h}{h + t} = \frac{0}{0 + 3} = 0.$$

However, is it reasonable to conclude that the coin never produces heads? This is where prior knowledge can play a crucial role. To address this limitation, we can use Laplace smoothing, which incorporates a prior into our estimate. The formula for **Laplace smoothing** is:

$$\phi = \frac{h + 1}{h + t + 2}$$

Thus,

$$\phi = \frac{h + 1}{h + t + 2} = \frac{0 + 1}{0 + 3 + 2} = \frac{1}{5}$$

This ensures $\phi > 0$ even when no heads are observed.

Large Data Example:

Now, let's consider a larger dataset where:

$$h = 100, \quad t = 60$$

Using the standard method, the estimate for ϕ is:

$$\phi = \frac{100}{100 + 60} = \frac{100}{160} = 0.625$$

Applying Laplace smoothing gives us:

$$\phi = \frac{100 + 1}{100 + 60 + 2} = \frac{101}{162} \approx 0.623$$

Both estimates are quite similar because the data sizes dominate the influence of the $+1$ in the numerator and the $+2$ in the denominator.

Note:

- In cases with limited trials (such as three), one might question the reliability of a ϕ estimate of 0. It's possible, through chance, to observe only tails. A fair coin will typically produce heads as well, so one should consider this when analyzing results.
- Laplace smoothing allows our estimation to be less extreme. By giving our estimate a slight adjustment (adding 1 to heads and 2 to the total count), we acknowledge that while our data may show tails, we still account for the possibility of heads existing, leading to a more reasonable estimate.
- Laplace smoothing is particularly useful when data is sparse. As the amount of data increases, the influence of Laplace smoothing diminishes, and the estimates converge toward the true proportions observed in the data. When large datasets are available, the raw counts become more trustworthy.

Laplace smoothing parameters

From our Naive Bayes classifier, we know that

$$\begin{aligned} \phi_{j|y=1} &= \frac{\sum_{i=1}^{n} \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = 1\}}{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 1\}} \\ \phi_{j|y=0} &= \frac{\sum_{i=1}^{n} \mathbb{1}\{x_j^{(i)} = 1, y^{(i)} = 0\}}{\sum_{i=1}^{n} \mathbb{1}\{y^{(i)} = 0\}} \end{aligned}$$

With Laplace smoothing, the estimations of the parameters adjust as follows:

$$\begin{aligned}\phi_{j|y=1} &= \frac{1+\sum_{i=1}^{n} \mathbb{1}\{x_j^{(i)}=1, y^{(i)}=1\}}{2+\sum_{i=1}^{n} \mathbb{1}\{y^{(i)}=1\}} \\ \phi_{j|y=0} &= \frac{1+\sum_{i=1}^{n} \mathbb{1}\{x_j^{(i)}=1, y^{(i)}=0\}}{2+\sum_{i=1}^{n} \mathbb{1}\{y^{(i)}=0\}}\end{aligned}$$

From the case where the feature does not appear in the dataset in equations (1, 2). With Laplace smoothing, however, the estimation changes:

$$\begin{aligned}\phi_{2|y=1} &= \frac{0+1}{\#\text{ of positive examples}+2} > 0 \\ \phi_{2|y=0} &= \frac{0+1}{\#\text{ of negative examples}+2} > 0\end{aligned}$$

Extension to Multiple Classes

If extending this idea to multiple classes (such as rolling a die):

Given $z \in \{0, 1, \cdots, k-1\}$, the probability without smoothing is:

$$p(z=j) = \frac{\sum_{i=1}^{n} \mathbb{1}\{z^{(i)}=j\}}{n}$$

After applying Laplace smoothing, it would be:

$$p(z=j) = \frac{1+\sum_{i=1}^{n} \mathbb{1}\{z^{(i)}=j\}}{n+k}$$

This adjustment adds uniformity and reliability to estimates across various categories.

Application of Laplace Smoothing

When applied to our classification problem (such as spam filtering), the numerator counts the occurrences of specific features in positive examples, while the denominator represents the total number of positive examples. This ensures no probability estimate is exactly zero, addressing the issue of extreme estimates (like for the 'aardvark') effectively.

Laplace smoothing effectively makes sure that if a class has not appeared in the dataset, it won't lead to a zero estimate:

- It adjusts all estimates to be greater than zero.
- Helps ensure that models remain robust, particularly in cases with few examples.

Laplace smoothing provides a means to moderate the estimates derived from limited data by introducing a small constant. It ensures that all feature probabilities are non-zero, enhancing the model's ability to generalize from sparse datasets.

V. Summary

In this lecture, we have explored essential concepts related to predictive modeling and the challenges faced in calculating probabilities using Naive Bayes classifiers. Here are the key takeaways:

- We focused on the computation of conditional probabilities, particularly $P(y = 1|x)$, and how classifications depend on these probabilities.
- The zero-probability problem arises when features are unseen in the training data, leading to indeterminate forms such as $\frac{0}{0}$. This limitation impairs effective classification.
- We introduced **Laplace smoothing** as a solution to the zero-probability problem. By adding a small constant (usually 1) to counts, we ensured that no probability estimate is exactly zero.
- Laplace smoothing not only mitigates the issues arising from sparse data but also provides more reliable probability estimates that can handle unseen features.
- We demonstrated Laplace smoothing with various examples, including flipping a coin and applying it within the context of Naive Bayes classifiers.
- Finally, we extended the concept of Laplace smoothing to scenarios involving multiple classes, thereby enhancing the robustness and versatility of our probabilistic models.

Module 4: Supervised Learning
Submodule: 4.8 Kernels
Lecture: Kernels

XCS229 Machine Learning

[Tengu Ma]

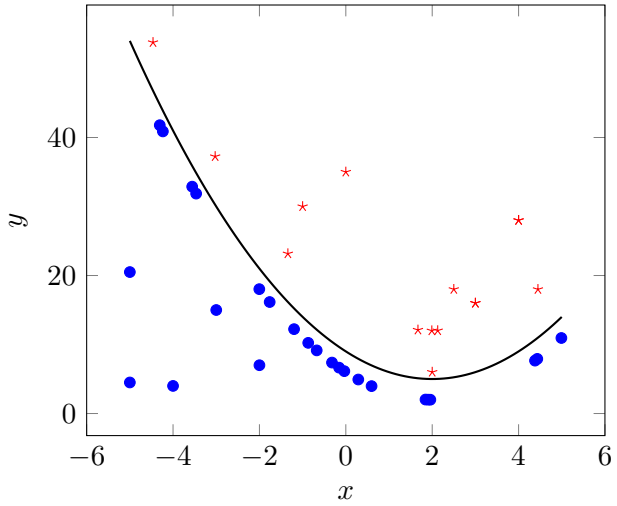
Contents

I. Introduction

In this lecture, we discuss the **kernel method**, a general technique for dealing with nonlinear relationships in data. We will explore how this method can be applied in supervised learning, particularly within discriminative algorithms, and will touch upon its application in unsupervised learning.

Lets consider a simple dataset in a regression setting where we have some data points denoted as (x, y). The aim is to fit a model that appropriately represents the relationship in this dataset. Linear regression might not be appropriate if the data is inherently nonlinear.

Non-linear Data Representation



II. Cubic Polynomial Fit

To deal with nonlinearity, we can use a cubic polynomial model:

$$h_\theta(x) = \theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0$$

where $x \in \mathbb{R}$.

Although $h_\theta(x)$ is nonlinear in x, it is linear in terms of the parameters θ. In linear regression, what really matters is that we are linear in our parameters θ because we are optimizing over the parameter space.

To turn our nonlinear model into a linear one, we can define a function $\phi(x)$ that maps x to a higher-dimensional vector:

$$\phi : \mathbb{R} \to \mathbb{R}^4$$

Specifically, we can define:

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

Then our model can be rewritten as:

$$\begin{aligned} h_\theta(x) &= \begin{bmatrix} \theta_0 & \theta_1 & \theta_2 & \theta_3 \end{bmatrix} \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \\ &= \theta^T \phi(x) \end{aligned}$$

Then $h_\theta(x)$ is linear in θ and $\phi(x)$.

Previously, we had a one-dimensional input x. Now, by using $\phi(x)$, we have transformed it into a four-dimensional vector. We can now redefine the dataset with new inputs as:

$$\begin{aligned} &\left\{\left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \ldots, \left(x^{(n)}, y^{(n)}\right)\right\} \\ \xrightarrow{\text{new data set}} &\left\{\left(\phi\left(x^{(1)}\right), y^{(1)}\right), \left(\phi\left(x^{(2)}\right), y^{(2)}\right), \ldots, \left(\phi\left(x^{(n)}\right), y^{(n)}\right)\right\} \end{aligned}$$

where $x^{(i)}, y^{(i)} \in \mathbb{R}$.

III. Implementing Linear Regression

We can implement linear regression on the new dataset

$$\left\{\left(\phi\left(x^{(i)}\right), y^{(i)}\right)\right\}_{i=1}^{n}$$

using gradient descent. The update rule for gradient descent is given by:

$$\theta \leftarrow \theta + \alpha \cdot \nabla J(\theta)$$

Where $J(\theta) = \frac{1}{2}\sum_{i=1}^{n}\left(y^{(i)} - \theta^T\phi(x^{(i)})\right)^2$. The gradient of the cost function can be expressed as:

$$\nabla J(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \theta^T\phi(x^{(i)})\right)\phi(x^{(i)})$$

We can rewrite it in a more iterative form:

$$\theta := \theta + \alpha\sum_{i=1}^{n}\left(y^{(i)} - \theta^T\phi(x^{(i)})\right)\phi(x^{(i)})$$

In essence, we are reiterating the linear regression procedures we learned previously. You just write down an algorithm for this new dataset and implement the linear regression algorithm on it.

Although we are repeating concepts from the first two weeks, it is essential to revisit them, as we will reuse these ideas to demonstrate something else, specifically, the **kernel trick**.

To perform linear regression on this new dataset, we will use gradient descent. We compare the label with the model, for which we have $\phi(x^{(i)})$.

Recall that with standard linear regression, the approach assumes that the model is:

$$h_\theta(x) = \theta^T x$$

Now, we replace $x^{(i)}$ with $\phi(x^{(i)})$.

Similarly, the corresponding stochastic gradient descent update rule is

$$\theta := \theta + \alpha \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)})$$

In the standard case, both instances would refer directly to $x^{(i)}$. However, in our case, we are now utilizing the transformed feature version $\phi(x^{(i)})$.

Note that the dimensions of θ and $\phi(x)$ are different. While x is generally of dimension d, $\phi(x)$ maps $\mathbb{R}^d$ to some new dimension p ($\phi(x) : \mathbb{R}^d \to \mathbb{R}^p$).

Dimension of $\phi(x)$ is p.

This means θ must also be in the same space since it will be a linear combination of transformed inputs. Thus, we are updating in this p-dimensional space rather than the original x-dimensional space.

It's worth noting that this mapping is often referred to as a feature map. The function ϕ maps from $\mathbb{R}^d$ to $\mathbb{R}^p$, and in this context, $\phi(x)$ is often labeled as features.

While reviewing different papers on the subject, you might find that x is referenced as attributes to distinguish from features. The terms may vary slightly depending on the author, but the core understanding remains consistent.

In most cases, if a linear function is applied to a variable, it is considered a feature.

Note:

- While everything appears clear when substituting $x^{(i)}$ with $\phi(x^{(i)})$, we must recognize that there are still considerations to be made. In some scenarios, this approach works well, as seen in the homework example where you were requested to implement a similar process.
- However, the task was to utilize a different algorithm, specifically one that leveraged the inverse of the time-summed matrices in gradient calculations, showing that we can adopt multiple methods to tackle the same problem.

IV. Challenges with High-Dimensional Feature Mapping

As we apply our existing algorithm to the new dataset, we need to consider some computational efficiency issues that arise, particularly with high-dimensional feature maps.

For instance, lets consider $x \in \mathbb{R}^d$ and let $\phi(x)$ be the vector that captures all the monomials of x with degrees less than or equal to 3. The vector representation can be illustrated as follows:

$$\phi(x) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \\ x_1^2 \\ x_1x_2 \\ x_1x_3 \\ \vdots \\ x_1x_d \\ x_2x_1 \\ \vdots \\ x_d^2 \\ x_1^3 \\ x_1x_2^2 \\ \vdots \\ x_d^3 \end{bmatrix} \begin{matrix} \} 1 \\ \\ \\ \\ \} d \\ \\ \\ \\ \\ \} d^2 \\ \\ \\ \\ \\ \} d^3 \\ \\ \\ \end{matrix}$$

Every degree-3 polynomial can be expressed as $\theta^T \phi(x)$, where $\phi(x) \in \mathbb{R}^p$ and $p = 1 + d + d^2 + d^3$. For example, if $d = 10^3$, then:

$$p = 1 + 10^3 + (10^3)^2 + (10^3)^3 = 10^9$$

The update rule for gradient descent becomes:

$$\theta := \theta + \alpha \underbrace{\sum_{i=1}^{n} \left(y^{(i)} - \underbrace{\theta^T \phi(x^{(i)})}_{O(p)} \right) \phi(x^{(i)})}_{O(np) \text{ total operations}}$$

Due to the high dimensionality of $\phi(x)$, where p can be extremely large (e.g., 10^9), we encounter significant computational challenges. Specifically, each

iteration of gradient descent requires $O(p)$ operations for the inner product and an additional $O(np)$ operations for the entire update step.

This implies that if p is on the order of 10^9 and we have n samples, then the total number of required operations can become prohibitive in practice.

The objective of the next lecture is to introduce the **kernel trick**, which allows us to implement nonlinear models more efficiently without explicitly computing the high-dimensional feature mappings. The final algorithm will maintain the same functionality as the direct implementation but will be considerably faster and more efficient.

Machine learning is fundamentally about computational efficiency, and as we strive for better algorithms, understanding that computational aspects are crucial for applying our methods to large datasets is vital.

V. Summary

In this lecture, we explored the concept of the **kernel method** as a powerful technique for handling nonlinear relationships in data, particularly in the context of supervised learning. Here are the key takeaways:

- **Transformation of Inputs:** We illustrated how nonlinear functions can be handled by transforming inputs into higher-dimensional spaces using the feature mapping function $\phi(x)$.
- **Cubic Polynomial Fit:** We discussed the cubic polynomial model as an example, demonstrating that even nonlinear models can be expressed in terms of linear parameters when using an appropriate feature mapping.
- **Gradient Descent in High Dimensions:** We learned how to apply gradient descent to the new transformed dataset, highlighting the importance of retaining linearity in the parameter space while dealing with nonlinearity in the input space.
- **Challenges with High Dimensions:** The lecture addressed the computational challenges associated with high-dimensional feature mappings, illustrating how high dimensionality can lead to inefficiencies in algorithm performance.
- **Importance of Computational Efficiency:** A recurring theme was the significance of computational efficiency in machine learning, emphasizing that effective algorithms must balance both statistical accuracy and computational feasibility.

Module 4: Supervised Learning
Submodule: 4.8 Kernels
Lecture: Computational Efficiency of Kernels

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this lecture, we explore the **kernel method** to speed up the algorithm under consideration.

II. Representing θ

For simplicity, at the beginning time step (i.e., $t = 0$ iteration) we initialize:

$$\theta^{(0)} = 0$$

We start by asserting that the vector θ can always be expressed as a linear combination of features:

$$\theta = \sum_{i=1}^{n} \beta_i \phi(x^{(i)})$$

for some $\beta_1, \ldots, \beta_n \in \mathbb{R}$. (No condition is needed if $\theta^0 = 0$.) This representation implies that we need only store the scalars β_i instead of the entire vector θ. β is now our "surrogate" for θ.

Let p be the dimensionality of θ which is significantly larger than n (i.e., $p \gg n$). This reduction from p to n parameters helps minimize storage and computational overhead.

Induction Proof for the Representation of θ

We will demonstrate how the vector θ can be represented as a linear combination of feature vectors using an induction proof.

- Base Case at Iteration 0: At iteration 0, we initialize:

$$\theta^{(0)} = 0 = \sum_{i=1}^{n} 0 \cdot \phi(x^{(i)})$$

  Here, all coefficients $\beta_i^{(0)}$ are zero, which affirms that θ is indeed a linear combination of the feature vectors, as required.

- Step for Iteration 1: At iteration 1, we update θ according to:

$$\begin{aligned}
\theta^{(1)} &:= \theta^{(0)} + \alpha \sum_{i=1}^{n} \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)}) \\
&= 0 + \alpha \sum_{i=1}^{n} y^{(i)} \phi(x^{(i)}) \\
&= \sum_{i=1}^{n} \underbrace{\alpha y^{(i)}}_{\beta_i \text{ at iteration 1}} \phi(x^{(i)})
\end{aligned}$$

  where θ^T in the first row is at $t = 0$. This establishes that $\theta^{(1)}$ is a linear combination of the feature vectors.

- Induction Hypothesis: Assume at iteration t, the representation holds:

$$\theta^{(t)} = \sum_{i=1}^{n} \beta_i \phi(x^{(i)})$$

- Induction Step at Iteration $t + 1$: Show this holds for iteration $t + 1$:

$$\begin{aligned}
\theta^{(t+1)} &:= \theta^{(t)} + \alpha \sum_{i=1}^{n} \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)}) \\
&= \sum_{i=1}^{n} \beta_i \phi(x^{(i)}) + \alpha \sum_{i=1}^{n} \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right) \phi(x^{(i)}) \\
&= \sum_{i=1}^{n} \underbrace{(\beta_i + \alpha \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right))}_{\text{new } \beta_i} \phi(x^{(i)})
\end{aligned}$$

where θ^T in the first row is at iteration t. This equation indicates that at iteration $t+1$, θ remains a linear combination of the feature vectors with updated coefficients β_i.

By induction, we showed that at each iteration, θ can be represented as:

$$\theta = \sum_{i=1}^{n} \beta_i \phi(x^{(i)})$$

This representation holds true for all iterations, demonstrating that the algorithm consistently maintains the linear combination format as it updates through iterations. As such, we only need to store n parameters as opposed to n parameters.

In order to implement this, we derive the update rule of the coefficients $\beta_1, \ldots, \beta_n$. Using the equation above, we see that the new β_i depends on the old one via:

$$\beta_i := \beta_i + \alpha \left(y^{(i)} - \theta^T \phi(x^{(i)}) \right)$$

Here we still have the old θ on the RHS of the equation. Replacing θ using $\theta = \sum_{j=1}^{n} \beta_j \phi(x^{(j)})$ gives:

$$\forall i \in \{1, \ldots, n\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^{n} \beta_j \phi(x^{(j)})^T \phi(x^{(i)}) \right)$$

We often rewrite

$$\phi(x^{(j)})^T \phi(x^{(i)})$$

as

$$\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$$

to emphasize that it's the inner product of the two feature vectors. Thus

$$\forall i \in \{1, \ldots, n\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^{n} \beta_j \langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle \right)$$

Significance: This allows us to update β_i without direct dependence on θ. Viewing β_i as the new representation of θ, we have successfully translated the batch gradient descent algorithm into an algorithm that updates the value of β iteratively.

It may now appear that at every iteration, we still need to compute the values of $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ for all pairs of i, j, each of which may take roughly $O(p)$ operations. However, two important properties come to rescue:

- **Pre-computation:** We can pre-compute the pairwise inner products $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ for all pairs of i, j before the loop starts, meaning that we only need to do this once at the very beginning.

- **Efficient Feature Map Calculation:** For the feature map ϕ, computing $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$ can often (but not always) be efficient and does not necessarily require computing $\phi(x^{(i)})$ explicitly.

Using the ϕ from the previous lecture and choosing x and z as arbitrary indicies, we begin with:

$$\langle \phi(x), \phi(z) \rangle = \begin{bmatrix} 1 & x_1 & x_2 & \cdots & x_1^2 & x_1x_2 & \cdots & x_2x_1 & \cdots & x_1^3 & \cdots \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_1^2 \\ x_1x_2 \\ x_1x_3 \\ \vdots \\ x_2x_1 \\ \vdots \\ x_1^3 \\ x_1^2x_2 \\ \vdots \end{bmatrix}$$

This is because:

$$\begin{aligned} \langle \phi(x), \phi(z) \rangle &= 1 + \sum_{i=1}^{d} x_i z_i + \sum_{i,j \in \{1,\ldots,d\}} x_i x_j z_i z_j + \sum_{i,j,k \in \{1,\ldots,d\}} x_i x_j x_k z_i z_j z_k \\ &= 1 + \sum_{i=1}^{d} x_i z_i + \left(\sum_{i=1}^{d} x_i z_i \right) \left(\sum_{i=1}^{d} x_i z_i \right) \\ &\quad + \left(\sum_{i=1}^{d} x_i z_i \right) \left(\sum_{i=1}^{d} x_i z_i \right) \left(\sum_{i=1}^{d} x_i z_i \right). \end{aligned}$$

Therefore, we can express this as:

$$\langle \phi(x), \phi(z) \rangle = 1 + \sum_{i=1}^{d} x_i z_i + \left(\sum_{i=1}^{d} x_i z_i \right)^2 + \left(\sum_{i=1}^{d} x_i z_i \right)^3$$

Thus, to compute $\langle \phi(x), \phi(z) \rangle$, we can first compute $\langle x, z \rangle$ with $O(d)$ time and then take a constant number of operations to compute

$$\langle \phi(x), \phi(z) \rangle = 1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3 \tag{1}$$

You might wonder if you really saved any time. Not yet because if you want to compute a new β from the old β, you still have to conduct this

inner product. This inner product between two p-dimensional vectors still takes $O(p)$ time. You still have the sum over n indices, so it's still $O(n \cdot p)$. Therefore, you haven't really saved much since ϕ still shows up here.

Here is where the magic happens. The inner product can be preprocessed for all pairs of i, j. This means you can compute it once and not have to recompute it every time since this quantity does not really change over time in your algorithm. By "over time," we mean as your algorithm is running this quantity doesn't change at all.

Additionally, the other important thing is that oftentimes you can compute this inner product faster than you thought. The naive implementation would be to just compute this factor and that factor and take the inner product, but in many cases you can do some math to speed it up.

This can be computed without even evaluating ϕ explicitly. If you have to evaluate each of these factors, you already wasted $O(p)$ time. However, sometimes you can do some math to not even evaluate ϕ.

The reason you can save on degrees of freedom, despite having p degrees of freedom in your θ, is because when your dataset is not as large as p, you cannot utilize all of them. You are not fundamentally using all the dimensionality or all the degrees of freedom in θ. If you have n samples, the maximum degrees of freedom you can achieve is n samples, and that's why you can save without any approximation.

This approach is really just a reimplementation of the same algorithm. We didn't lose any information; it's not like you're doing any approximation at all.

Now, let's say this clearly: there is a notation that people use to call this a so-called **kernel function**. The kernel function allows us to compute the inner product in a high-dimensional space without explicitly mapping data points into that space, making many algorithms more efficient and scalable.

III. Kernel Function

We define the **Kernel** corresponding to the feature map ϕ as a function that maps $\mathcal{X} \times \mathcal{X} \to \mathbb{R}$ satisfying:

$$K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle$$

$\mathcal{X}$ is the space of the input x. In our running example, $\mathcal{X} = \mathbb{R}^d$.

The final kernel algorithm is as follows:

Algorithm 1 Kernel Algorithm

1. Compute all the values $K(x^{(i)}, x^{(j)}) \triangleq \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$ using equation (1) for all $i, j \in \{1, \ldots, n\}$. Set $\beta := 0$ with $\beta \in \mathbb{R}^n$.

2. **Loop:**

$$\forall i \in \{1, \ldots, n\}, \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^{n} \beta_j K(x^{(i)}, x^{(j)}) \right)$$

In vector notation, letting K be an $n \times n$ matrix with $K_{ij} = K(x^{(i)}, x^{(j)})$, we have

$$\beta := \beta + \alpha(\vec{y} - K\beta)$$

Now, lets analyze the runtime for this particular polynomial feature.

- For computing K, we are using a formula that requires $O(d)$ time for each pair. Therefore, for all pairs (i, j), the total is $O(n^2 d)$.
- The loop updates each β_i, thus incurs $O(n^2)$ per iteration.

It's significant that there's no p involved anymore; it's just n and d. If n is small enough, particularly when $n^2 < p$, this is advantageous. Previously, each iteration took $n \times p$, but now it takes n^2. This method becomes especially useful when p is large compared to n, as in cases where p could be exponentially larger, such as d^3.

In conclusion, while the total runtime reflects $O(n^2 d) + O(n^2 T)$ where T is the number of iterations, the challenge remains in estimating T. It can vary significantly based on the specific problem. To make a meaningful comparison, one must generally consider that the absence of p implies a potential reduction in complexity and thus a more efficient algorithm, especially when p is large.

IV. Summary

In this lecture, we have discussed several key points regarding kernels and their computational efficiency:

- Kernels provide a method to compute inner products in high-dimensional spaces without explicitly mapping data points into that space.
- We have derived the representation of θ as a linear combination of feature vectors, emphasizing the importance of the coefficients β.

- The use of kernels allows for significant computational savings, particularly when transitioning from a high-dimensional space p to a lower-dimensional representation n.
- Pre-computation of inner products between feature vectors enhances the efficiency of algorithms using kernels.
- Our final kernel algorithm demonstrates how to update coefficients iteratively, resulting in a more efficient implementation of machine learning algorithms.

Module 4: Supervised Learning
Submodule: 4.8 Kernels
Lecture: Designing Kernels

XCS229 Machine Learning

[Tengu Ma]

Contents

I. Introduction

In this lecture, we explore the concept of kernels in machine learning. The focus will be on how to compute the inner product of features and the implications of this for algorithms. We start by understanding that we can often define our methods directly in terms of the kernel function rather than the feature mapping.

II. Understanding Kernels

We start with the observation that the inner product of features is all that matters for certain algorithms. Thus, the focus can shift from defining the feature mapping ϕ to defining the kernel function K. The core idea is that

we can work directly with the kernel function without needing to know the explicit form of ϕ.

We do not need the explicit form of ϕ as long as K is a valid kernel. However, it is essential to ensure that the kernel function satisfies certain properties so that the algorithm can be effective.

Test Time Computation

At test time, when given a new data point x we compute $\theta^T \phi(x)$ as:

$$\begin{aligned} \theta^T \phi(x) &= \left(\sum_i^n \beta \phi(x^{(i)}) \right)^T \phi(x) \\ &= \sum_i^n \beta_i K(x^{(i)}, x) \end{aligned}$$

If the kernel function K can be computed efficiently, then we do not need to compute ϕ explicitly. Note that $x^{(i)}$ is from the training set, so at test time we still need to look at the training examples. Typically you only need to store θ and the training examples can be ignored, but not here.

$$\text{If you design a good } \phi \Longleftrightarrow \text{designing a good } K$$

III. Kernel Matrix and Validity

We start with the notation for data points and kernel matrices:

$$x^{(1)}, \cdots, x^{(n)} \text{ data points}$$

Let $K \in \mathbb{R}^{n \times n}$ be the kernel matrix defined as:

$$K_{ij} = K(x^{(i)}, x^{(j)})$$

Having K represent both a matrix (the LHS) as well as a function (the RHS) can be confusing, so this is worth noting.

Theorem 1. *K is a valid kernel function if and only if for all $x^{(1)}, \cdots, x^{(n)}$, the kernel matrix K is positive semi-definite (PSD).*

The proof can be approached in two directions:

- To show that if the kernel K is valid, then the corresponding kernel matrix is PSD follows from the definition:

$$\langle \phi(x), \phi(z) \rangle \Longrightarrow \text{ the matrix is PSD}$$

- The reverse direction requires more effort and careful consideration given the regularity conditions required for the kernel functions involved.

Kernel Design Workflow

The typical workflow for creating a kernel function is:

1. Design the kernel function K.
2. Verify that the kernel is valid by checking whether the corresponding kernel matrix K is PSD (from theorem 1) or by constructing $\phi(\cdot)$.
3. Run the algorithm (called Kernel trick or kernelized) .

Kernel Trick

The kernel trick allows us to transform algorithms operating in the feature space in a way that we can use kernel functions instead of featuring mappings directly. This can be applied to various contexts, such as starting with linear regression or logistic regression and translating those algorithms into the kernel space.

The kernel trick means that an algorithm developed with feature mapping $\phi(x)$ can instead be expressed with kernel K.

Not all algorithms can be transformed using the kernel trick. For example, some algorithms, such as those using L1 regularization, may not be easily kernelized since they do not only rely on pairwise inner products.

The key requirement for kernelization is that the algorithm or function can be expressed as the inner products between pairs of data points. If an entire algorithm can be expressed as an operation solely relying on such pairs, it can potentially be kernelized.

While we primarily discussed one kernel, several other kernels can be created for specific applications. Some examples include polynomial kernels, Gaussian kernels, and other custom kernels designed based on the problem at hand.

IV. Examples of Kernel Functions

We can define several kernel functions, including:

$$K(x, z) \quad = \quad (\langle x, z \rangle + c)^k \, .$$

For any choices of c and k, this is a valid kernel. For example, with

$k = 2$, we have the feature mapping:

$$\phi(x) = \begin{bmatrix} c \\ \sqrt{2c}x_1 \\ \sqrt{2c}x_2 \\ \sqrt{2c}x_3 \\ \vdots \\ x_1^2 \\ \vdots \\ x_d^2 \end{bmatrix},$$

A common kernel function is the Gaussian kernel defined as:

$$\begin{aligned} K(x,z) &= \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right) \\ &= \langle \phi(x), \phi(z) \rangle \end{aligned}$$

For this kernel, $\phi(x)$ and $\phi(z)$ represent features that can become very complex and may require infinite dimensions to fully express.

Interpretation of Kernel Functions

- The inner product between two feature vectors can be understood as a measure of similarity between them. When x and z are similar, the kernel $K(x,z)$ gives a larger value, thus serving as a similarity metric.
- The kernel trick enables one to avoid directly computing infinite-dimensional features by replacing the inner product calculations with evaluations of the kernel function. This allows easier computations without explicitly defining the high-dimensional feature space.

However, it is essential to recognize the computational implications:

- The runtime complexity increases to $O(n^2)$ due to pairwise calculations, where n is the number of data points.
- This becomes prohibitive for very large datasets, as n^2 can grow very rapidly (e.g., for $n = 1,000,000$, it results in 10^{12} calculations).

Another significant limitation is that kernel methods often depend on handcrafted features, which require intentional design and can limit the method's effectiveness. Conversely, newer approaches in machine learning, like deep learning, automatically learn features from data. For a model:

$$\theta^T \phi_w(x),$$

where $\phi_w(x)$ is parameterized based on learned data features, demonstrates that modern methods often introduce learnable features that adapt to the dataset.

Finally, while kernel methods provide a powerful way to handle non-linear classification and regression, their dependence on handcrafted features and computational efficiency in large datasets has led to a shift towards learned feature representations in modern machine learning contexts.

V. Summary

In this lecture, we explored the concept of kernels in machine learning, focusing on their role in facilitated computation through inner products of features without explicitly knowing the feature mappings. Here are the key takeaways:

- **Kernels Over Feature Mappings:** The essential idea behind kernels is that algorithms can often be defined directly through kernel functions rather than the explicit feature mappings ϕ.
- **Validity of Kernels:** A kernel K is deemed valid if it is possible to express it in terms of inner products of mappings ϕ. The kernel matrix must be positive semi-definite (PSD).
- **Kernel Trick:** The kernel trick enables transforming algorithms to work in a higher-dimensional space without computing the features directly, aiding complex computations efficiently.
- **Examples of Kernels:** Various kernels, such as polynomial kernels and Gaussian (RBF) kernels, illustrate the versatility in kernel design. Each kernel function may correspond to a specific type of feature mapping and problem domain.
- **Computational Implications:** The substitution of explicit feature mapping for kernel functions generally increases computational overhead (to $O(n^2)$), which might become prohibitive for large datasets.
- **Shift Towards Learned Features:** The trend in modern machine learning is toward automatic feature learning (as seen in deep learning), which contrasts with earlier methods that relied heavily on handcrafted features through kernels.